ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«УФИМСКИЙ УНИВЕРСИТЕТ НАУКИ И ТЕХНОЛОГИЙ»
Институт среднего профессионального образования

Специальность:

09.02.07 Информационные системы и программирование

Квалификация:

Программист

МДК.11.01 Технология разработки и защиты баз данных

ПМ.11 «Разработка, администрирование и защита баз данных»

Курсовой проект

РАЗРАБОТКА И ЗАЩИТА БАЗЫ ДАННЫХ «ЖЕЛЕЗНОДОРОЖНЫЙ ВОКЗАЛ»

Научный руководитель:
преподаватель
_____ Старцева И.В.

Выполнил: студент(ка) 3 курса очной формы обучения группы ИСП(п)3523
_____ Тухватшин Р.А.

допущен
18.04.2026г.

УФА – 2026

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«УФИМСКИЙ УНИВЕРСИТЕТ НАУКИ И ТЕХНОЛОГИЙ»
Институт среднего профессионального образования

ЗАДАНИЕ

на курсовой проект по МДК.11.01 Технология разработки и защиты баз данных
ПМ.11 Разработка, администрирование и защита баз данных

Тема: Разработка и защита базы данных «железнодорожный вокзал»

студенту(ке) Тухватшину Рустаму Айдаровичу
курса 3 группы ИСП(п)3523 специальность 09.02.07 Информационные системы и программирование

1. Дата выдачи задания «14» февраля 2026 г.
2. Срок сдачи законченной работы «18» апреля 2026 г.
3. Перечень подлежащих разработке вопросов:

1. Проектирование базы данных
   1.1. Анализ и описание предметной области
   1.2. Построение инфологической (концептуальной модели) предметной области
   1.3. Построение даталогической модели БД
2. Создание базы данных в Microsoft SQL Server и управление учетными записями и ролями
   2.1. Создание базы данных, резервное копирование и восстановление базы данных
   2.2. Создание пользователей и ролей для доступа к серверу через утилиту Microsoft SQL Server Management Studio
3. Разработка таблиц и ограничений
4. Создание запросов
   4.1. Создание запросов на выборку
   4.2. Создание многотабличных запросов. Запросы на соединение
   4.3. Создание запросов на группировку и сортировку данных. Запросы на изменение. Использование встроенных функций.
5. Создание представлений

4. Календарный график выполнения работы:

Введение	«21» февраля 2026г.
Теоретическая часть	«14» марта 2026г.
Практическая часть	«10» апреля 2026г.
Заключение	«18» апреля 2026г.

Преподаватель: _____ Старцева И.В., преподаватель
(подпись, Ф.И.О., должность, уч. степень)

Задание принято к исполнению: «14» февраля 2026г.
Студент: _____ Тухватшин Р.А.

Содержание

Введение....4
1 Проектирование базы данных....6
1.1 Анализ и описание предметной области....6
1.2 Построение инфологической (концептуальной модели) предметной области....10
1.3 Построение даталогической модели БД....11
2 Создание базы данных в Microsoft SQL Server и управление учетными записями и ролями....13
2.1 Создание базы данных, резервное копирование и восстановление базы данных....15
2.2 Создание пользователей и ролей для доступа к серверу через утилиту Microsoft SQL Server Management Studio....17
3. Разработка таблиц и ограничений....19
4 Создание запросов....28
4.1 Создание запросов на выборку....28
4.2 Создание многотабличных запросов. Запросы на соединение....32
4.3 Создание запросов на группировку и сортировку данных. Запросы на изменение. Использование встроенных функций....40
5 Создание представлений....45
Заключение....49
Список использованных источников....51

Введение

В рамках данного проекта предполагается не только структурное проектирование, но и логическое обоснование выбора средств реализации, а также разработка модели безопасности данных, обеспечивающей разграничение прав доступа для различных категорий сотрудников вокзала (администраторов, кассиров, диспетчеров). Итогом работы станет целостный программный продукт, способный служить основой для автоматизации ключевых бизнес–процессов железнодорожного вокзала.

Актуальность разработки базы данных для железнодорожного вокзала определяется современными тенденциями цифровизации транспортной инфраструктуры и необходимостью эффективного управления большим объёмом информации, связанной с функционированием вокзала. В настоящее время в рамках стратегии развития ОАО «РЖД» на период до 2030 г. инициируется проект «Цифровая железная дорога», включающий идеи автоматизации работы вокзальных комплексов и концепцию «Умного вокзала». Автоматизация систем управления процессами жизнеобеспечения и обслуживания пассажиров через цифровые платформы рассматривается как ключевой фактор повышения качества и надёжности работы вокзала. В условиях непрерывных, массовых и разнородных информационных потоков, характерных для вокзала, интегрированная база данных становится необходимой для централизованного учёта сведений о движении поездов, персонале, расписании и продаже билетов.

Объектом исследования в рамках данного курсового проекта выступает железнодорожный вокзал как сложный инфраструктурный и информационный комплекс. Железнодорожный вокзал выполняет функции обслуживания пассажиров и организации движения поездов, объединяя материальные элементы (здания, пути, оборудование) и разнообразные информационные процессы (планирование расписаний, обмен данными между системами управления, учёт деятельности персонала и т.д.). Вокзал как система

характеризуется непрерывным поступлением разнотипной информации о прибытии и отправлении поездов, обслуживании пассажиров и операциях по продаже билетов, что предъявляет высокие требования к информационной организации и обработке данных. Для эффективного управления такими информационными потоками необходима централизованная система учёта и хранения данных, что обосновывает разработку специализированной базы данных.

Предметом исследования выступают информационные процессы и структуры данных, реализуемые в базе данных железнодорожного вокзала. Данная база данных предназначена для хранения и обработки сведений о ключевых объектах деятельности вокзала: о сотрудниках (их структурное расположение, должности, рабочие смены), подвижном составе (номера поездов, их составы, типы), расписаниях движения поездов (время отправления и прибытия, дни курсирования, маршруты), бригадах проводников, а также о реализации билетов и операциях продаж. Предмет исследования охватывает анализ информационных потоков между этими сущностями (например, назначение проводника на конкретный поезд в определённую дату или учёт продаж билетов по маршруту) и проектирование соответствующих структур данных, обеспечивающих целостность и непротиворечивость информации.

Цель курсового проекта: разработка и защита базы данных «Железнодорожный вокзал».

Задачи курсового проекта:

1) Анализ и описание предметной области.
2) Построение инфологической и даталогической моделей.
3) Разработка базы данных, создание пользователей и ролей для доступа к серверу.
4) Создание запросов и представлений.

1 Проектирование базы данных

1.1 Анализ и описание предметной области

Предметная область – это железнодорожный вокзал и связанные с ним бизнес–процессы: управление персоналом, формирование расписаний движения поездов, комплектование поездов экипажами и продажа билетов пассажирам. Предметная область отображает совокупность реальных объектов (поезда, работники, рейсы, пассажиры и т.д.), о которых хранится информация в системе. Сущность предметной области представляет собой результат абстрагирования реального объекта через выделение и фиксацию его свойств. Перед началом проектирования необходимо досконально изучить, какие данные нужны пользователям системы: как формируются графики движения поездов, какие роли есть у работников вокзала (машинисты, проводники, кассиры и др.), как регистрируются продажи билетов, какие статусы билетов (бронирован/продан) и т.д. Результатом анализа является список требований и сценариев использования (например, оформление билета, изменение расписания, назначение бригады на поезд), а также перечень главных объектов предметной области.

В результате проведенного анализа предметной области базы данных «Железнодорожный вокзал» легко перечислить основные сущности этой БД. Так как на физическом уровне сущности соответствует таблица, то просто перечислим основные таблицы БД.

Список сущностей представлен в таблице 1.

Таблица 1. Список сущностей

№	Название	Назначение
1	Работники вокзала	Информация о сотрудниках станции разных специальностей
2	Поезд	Информация о каждом локомотиве/поезде

Продолжение таблицы 1.

3	Расписание	Конкретный рейс поезда
4	Бригада	Состав экипажа поезда
5	Продажи билетов	Информация о продаже билетов пассажирам
6	Состав бригад	Информация о составе бригад

Для каждой таблицы (сущности) приведем описание ее атрибутов. Атрибут на физическом уровне – это колонки таблицы и выражает определенное свойство объекта.

Список атрибутов сущности «Работники вокзала» представлен в таблице 2.

Таблица 2 Атрибуты сущности «Работники вокзала»

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ПК (первичный ключ)	Код работника	Ключевое поле. Представляет собой первичный ключ. Это уникальное значение, соответствующее каждому работнику. Это например, может быть его табельный номер.
	Фамилия	
	Имя	
	Отчество	
	Дата рождения	
	Дата поступления на работу	
	стаж	
	должность	машинисты, диспетчеры, кассиры и др
	пол	
	адрес	
	город	
	телефон	

Список атрибутов сущности «Расписание движения поездов» представлен в таблице 3.

Таблица 3 атрибуты сущности «Расписание движения поездов».

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ПК (первичный ключ)	Номер рейса	Ключевое поле. Представляет собой первичный ключ. Это уникальное значение, соответствующее каждому рейсу.
ВК(внешний ключ)	Номер поезда	С помощью данного внешнего ключа будет определен поезд.
	дата отправления	
	время отправления	
	место отправления	
	дата прибытия	
	время прибытия	
	место прибытия	
	маршрут	начальный и конечный пункты назначения, основные узловые станции
	стоимость билета	

Список атрибутов сущности «Поезда» представлен в таблице 4.

Таблица 4 атрибуты сущности «Поезда».

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ПК (первичный ключ)	Номер поезда	Ключевое поле. Представляет собой первичный ключ. Это уникальное значение, соответствующее каждому поезду.
	год выпуска	
	кол–во вагонов	
	тип поезда	общий, скоростной, высокоскоростной

Список атрибутов сущности «Бригады поездов» представлен в таблице 5.

Таблица 5 атрибуты сущности «Бригады поездов»

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ПК (первичный ключ)	номер бригады	Ключевое поле. Представляет собой первичный ключ. Это уникальное значение, соответствующее каждой бригаде.

Продолжение таблицы 5.

ВК(внешний ключ)	Номер поезда	С помощью данного внешнего ключа будет определен поезд.

Список атрибутов сущности «Ведомость продажи билетов» представлен в таблице 6.

Таблица 6 атрибуты сущности «Ведомость продажи билетов»

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ПК (первичный ключ)	Номер билета	Ключевое поле. Представляет собой первичный ключ. Это уникальное значение, соответствующее каждому билету. Это например, может быть его табельный номер.
	дата и время продажи	
	ФИО пассажира	
	паспортные данные	машинисты, техники, проводники и обслуживающий персонал
ВК(внешний ключ)	номер рейса	
	Кол–во билетов	
	наличие льгот	пенсионеры, дети–сироты и т.д
	стоимость	

Список атрибутов сущности «Состав бригад» представлен в таблице 7.

Таблица 7 атрибуты сущности «Состав бригад»

Ключевое поле	**Наименование атрибута**	**Краткое описание**
ВК(внешний ключ)	Код работника	С помощью данного внешнего ключа будет определен сотрудник
ВК(внешний ключ)	Код бригады	С помощью данного внешнего ключа будет осуществлена связь с таблицей бригады поездов.

Представленный перечень сущностей и их атрибутов является результатом первичного анализа предметной области.

1.2 Построение инфологической (концептуальной модели) предметной области

Инфологическое (концептуальное) моделирование является ключевым этапом проектирования, так как позволяет описать предметную область на языке специалиста, без привязки к конкретной СУБД. Цель данного этапа – создание формализованной структуры, отражающей состав объектов и характер связей между ними.

Инфологическая модель представлена в соответствии с рисунком 1.

Рисунок 1 – Инфологическая модель базы данных

Список связей сущностей представлен в таблице 8.

Таблица 8 список связей сущностей.

№	Название связи	Сущности, участвующие в связи	Назначение
1	M:M	Работники вокзала – поезда	Каждая бригада состоит из нескольких работников, и один работник может состоять во многих бригадах

Продолжение таблицы 8.

3	1:M	Поезд – Расписание поездов	Один поезд участвует во многих рейсах
4	1:M	Расписание поездов – продажи билетов	Одному рейсу принадлежит много билетов

1.3 Построение даталогической модели БД

Даталогическая модель представляет собой проекцию объектов предметной области на среду конкретной базы данных. Здесь происходит «материализация» сущностей в таблицы с указанием типов данных, ключей и ограничений.

Даталогическая модель представлена в соответствии с рисунком 2.

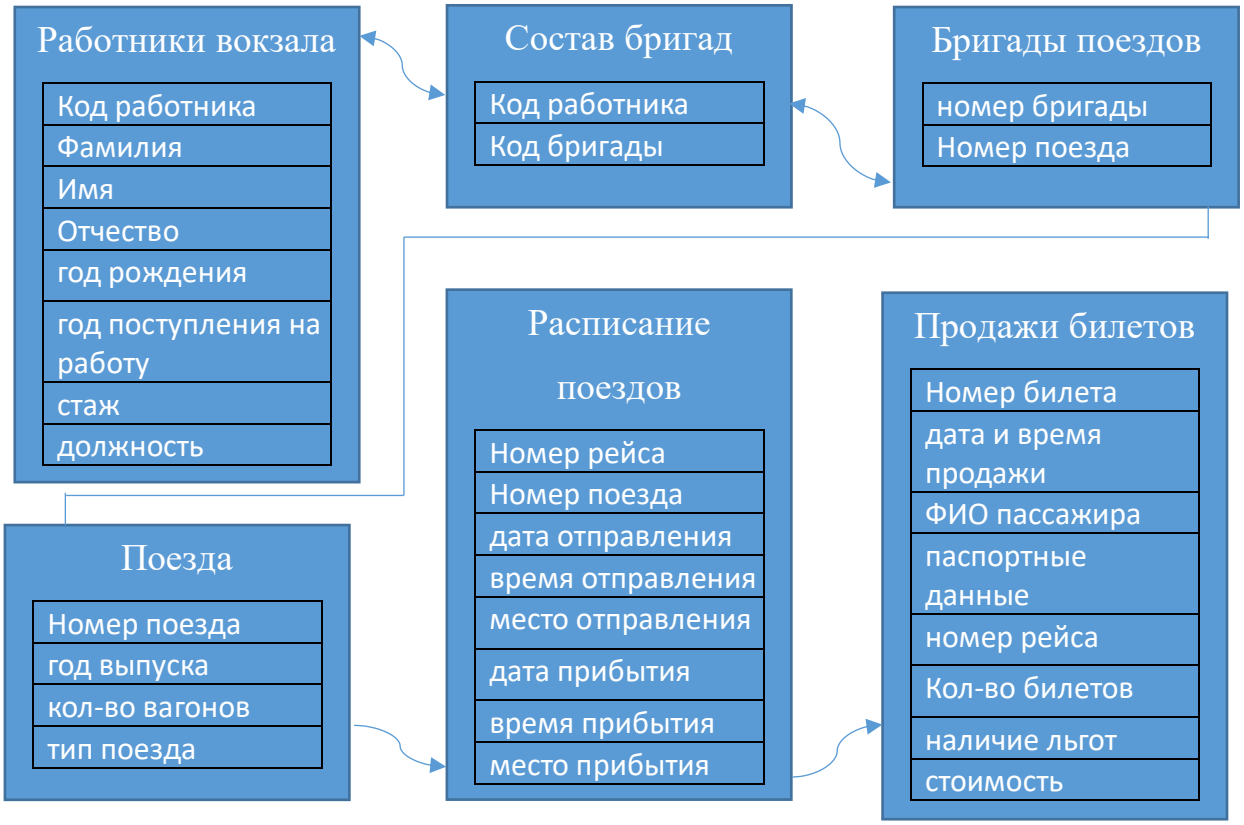

Рисунок 2 – Даталогическая модель базы данных

Даталогическая модель БД представляется в виде набора таблиц специальной формы, в которых указываются наименование атрибута, идентификатор, тип, длина, формат, ограничения.

Список атрибутов сущности «Работники вокзала» представлен в таблице 9.

Таблица 9 список атрибутов сущности «Работники вокзала».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	Код_работника	ID_Rabotnika	int	да	ПК (первичный ключ)
2	Фамилия	Familia	VARCHAR	да	
3	Имя	Name	VARCHAR	да	
4	Отчество	Otchestvo	VARCHAR	да	
5	Дата_рождения	Data_rozhdenia	date	да	
6	Дата_поступления _на_работу	Data_postuplenia	date	да	
7	Стаж	Stazh	расчетный	нет	
8	Должность	Dolzhnost	VARCHAR	да	
9	Пол	Gender	VARCHAR	да	
10	Адрес	Address	VARCHAR	нет	
11	Город	Gorod	VARCHAR	нет	
12	Телефон	Telefon	VARCHAR	да	уникальность

Список атрибутов сущности «Поезда» представлен в таблице 10.

Таблица 10 список атрибутов сущности «Поезда».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	Номер_поезда	ID_poezda	int	да	ПК (первичный ключ)
2	Дата_выпуска	Data_vipuska	date	да	
3	Количество_вагонов	Kol_vagonov	int	нет	
4	Тип_поезда	Tip_poezda	VARCHAR	да	

Список атрибутов сущности «Расписание поездов» представлен в таблице 311.

Таблица 11 список атрибутов сущности «Расписание поездов».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	Номер рейса	ID_reysa	int	да	ПК (первичный ключ)
2	Номер поезда	ID_poezda	int	да	ВК(вторичный ключ)
3	Дата отправления	Data_otprav	date	да	
4	Время отправления	Otprav_time	time	да	
5	место отправления	Mesto_prib	VARCHAR	да	
6	дата прибытия	Data_prib	date	да	
7	время прибытия	Prib_time	time	да	
8	место прибытия	Mesto_prib	VARCHAR	да	

Список атрибутов сущности «Ведомость продажи билетов» представлен в таблице 12.

Таблица 12 список атрибутов сущности «Ведомость продажи билетов».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	Номер билета	ID_bileta	int	да	ПК(первичный ключ)
2	дата и время продажи	Date_prodazhi	datetime	нет	
3	ФИО пассажира	FIO_passazhira	VARCHAR	да	
4	паспортные данные	Passport_info	VARCHAR	да	
5	номер рейса	ID_reysa	int	да	
6	Кол–во билетов	Kol_billetov	int	нет	
7	наличие льгот	Lgoti	VARCHAR	нет	
8	стоимость	stoimost	int	да	

Список атрибутов сущности «Состав бригад» представлен в таблице 13.

Таблица 13 список атрибутов сущности «Состав бригад».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	Код работника	ID_Rabotnika	int	да	ВК(вторичный ключ)
2	Номер бригады	ID_brigadi	int	да	ВК(вторичный ключ)

Список атрибутов сущности «Бригады» представлен в таблице 14.

Таблица 14 список атрибутов сущности «Бригады».

№	название	идентификатор	Тип	Не пусто	Ограничение
1	номер бригады	ID_brigadi	int	да	ПК (первичный ключ)
2	Номер поезда	ID_poezda	int	да	ВК(вторичный ключ)

В таблицах 9–14, приведена детальная структура каждой таблицы спроектированной базы данных «Железнодорожный вокзал». Для каждой таблицы определен первичный ключ (ПК), обеспечивающий уникальность записей, а также внешние ключи (ВК), необходимые для организации связей между таблицами и поддержания ссылочной целостности.

2 Создание базы данных в Microsoft SQL Server и управление учетными записями и ролями

2.1 Создание базы данных, резервное копирование и восстановление базы данных

Создание базы данных осуществлялось средствами Microsoft SQL Server с использованием языка T–SQL в среде SQL Server Management Studio (SSMS). Для создания базы данных была выполнена команда CREATE DATABASE, в которой указывались имя базы данных, пути к файлам данных и журнала транзакций, начальные размеры файлов, максимально допустимые размеры и шаг автоматического приращения.

Основной файл данных предназначен для хранения таблиц, индексов и прочих объектов базы данных. Файл журнала транзакций фиксирует все изменения, вносимые в базу данных, и служит основой для обеспечения целостности данных и возможности восстановления. Начальный размер файла данных задан равным 50 МБ с максимальным ограничением 500 МБ и шагом прироста 10 МБ; файл журнала инициализируется размером 10 МБ с максимумом 100 МБ и шагом 5 МБ.

Команда создания базы данных представлена в соответствии с рисунком 3.

```
CREATE DATABASE ZheleznodorozhniyVokzal
ON PRIMARY
(
    NAME = 'ZheleznodorozhniyVokzal',
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\ZheleznodorozhniyVokzal.mdf',
    SIZE = 50MB,
    MAXSIZE = 500MB,
    FILEGROWTH = 10MB
)
LOG ON
(
    NAME = 'ZheleznodorozhniyVokzal_log',
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\ZheleznodorozhniyVokzal_log.ldf',
    SIZE = 10MB,
    MAXSIZE = 100MB,
    FILEGROWTH = 5MB
);
```

Рисунок 3 – Создание базы данных ZheleznodorozhniyVokzal

После успешного создания базы данных было выполнено её резервное копирование. Резервное копирование является обязательной процедурой администрирования, обеспечивающей защиту данных от потери вследствие аппаратных сбоев, ошибок пользователей или программных неисправностей. Для создания резервной копии применялась команда BACKUP DATABASE, с указанием пути сохранения файла резервной копии, а также параметра WITH FORMAT, который инициализирует новый набор носителей и записывает первую резервную копию на этот носитель.

Файл резервной копии сохраняется в указанный каталог и содержит полный снимок состояния базы данных на момент выполнения операции. Команда резервного копирования представлена в соответствии с рисунком 4.

```
BACKUP DATABASE ZheleznodorozhniyVokzal
TO DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\ZheleznodorozhniyVokzal_backup.bak'
WITH FORMAT,
     MEDIANAME = 'SQLServerBackup',
     NAME = 'Резервная копия БД Железнодорожный вокзал',
     DESCRIPTION = 'Полная резервная копия базы данных ZheleznodorozhniyVokzal';
GO
```

Рисунок 4 – Создание резервной копии базы данных

При необходимости восстановление базы данных из резервной копии выполняется с помощью команды RESTORE DATABASE. В команде указывается источник файл резервной копии, а также параметр WITH REPLACE, позволяющий перезаписать существующую базу данных. Дополнительно задаются пути для восстанавливаемых файлов данных и журнала транзакций с помощью директивы MOVE, что позволяет гибко управлять расположением файлов при восстановлении на другом сервере или в другой директории. После выполнения операции резервное копирование можно проверить с помощью команды RESTORE VERIFYONLY, чтобы убедиться в целостности резервного файла. Процедура резервирования рекомендуется проводить регулярно, особенно перед выполнением обновлений или изменений структуры базы данных. Это позволяет обеспечить быстрое восстановление системы при возникновении непредвиденных ситуаций.

2.2 Создание пользователей и ролей для доступа к серверу через утилиту Microsoft SQL Server Management Studio

Для разграничения прав доступа к таблицам базы данных были определены три должностных лица: администратор, кассир и диспетчер. Такой подход позволяет распределить обязанности между сотрудниками вокзала и ограничить доступ к данным в зависимости от выполняемых функций.

Администратор обладает набором прав на работу с базой данных: просмотр, добавление, изменение и удаление записей во всех таблицах. Кассир получает права на работу только с данными, связанными с продажей билетов и расписанием рейсов: просмотр таблиц, добавление новых записей о продаже билетов и редактирование отдельных полей, необходимых для оформления билета. Диспетчер имеет доступ к таблицам, связанным с поездами, бригадами и расписанием, и может просматривать и корректировать сведения о рейсах без возможности изменять данные о продажах.

Первым шагом были созданы роли и выданы им привилегии с помощью команд CREATE ROLE и GRANT. Соответствующие команды представлены в соответствии с рисунками 5–7.

```
CREATE ROLE admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Rabotniki TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Poezda TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Brigadi TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Sostav_Brigad TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Raspisanie TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Prodazhi TO admin;
```

Рисунок 5 – Создание роли admin

```
CREATE ROLE kassir;
GRANT SELECT ON dbo.Raspisanie TO kassir;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Prodazhi TO kassir;
```

Рисунок 6 – Создание роли kassir

```sql
CREATE ROLE dispetcher;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Poezda TO dispetcher;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Brigadi TO dispetcher;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Sostav_Brigad TO dispetcher;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Raspisanie TO dispetcher;
```

Рисунок 7 – Создание роли dispetcher

Вторым шагом для каждой роли были созданы учётная запись на уровне сервера (CREATE LOGIN) и пользователь базы данных (CREATE USER). Пользователи были включены в соответствующие роли с помощью команды ALTER ROLE ... ADD MEMBER. Соответствующие команды представлены в соответствии с рисунком 8.

```sql
CREATE LOGIN admin_vokzal
WITH PASSWORD = 'Admin12345!';
CREATE LOGIN kassir_vokzal
WITH PASSWORD = 'Kassir12345!';
CREATE LOGIN dispetcher_vokzal
WITH PASSWORD = 'Dispetcher12345!';

CREATE USER admin_vokzal FOR LOGIN admin_vokzal;
CREATE USER kassir_vokzal FOR LOGIN kassir_vokzal;
CREATE USER dispetcher_vokzal FOR LOGIN dispetcher_vokzal;

ALTER ROLE admin ADD MEMBER admin_vokzal;
ALTER ROLE kassir ADD MEMBER kassir_vokzal;
ALTER ROLE dispetcher ADD MEMBER dispetcher_vokzal;
```

Рисунок 8 – Создание ролей, логинов, пользователей и назначение привилегий

Таким образом, в среде Microsoft SQL Server Management Studio была реализована модель разграничения доступа, основанная на использовании ролей и учетных записей пользователей. Данный подход обеспечивает безопасность данных, позволяет ограничить доступ сотрудников только к тем таблицам и операциям, которые необходимы им по должностным обязанностям, а также упрощает последующее администрирование базы данных.

3. Разработка таблиц и ограничений

Разработка структуры базы данных «Железнодорожный вокзал» выполнялась в среде Microsoft SQL Server Management Studio с использованием языка T–SQL. В соответствии с даталогической моделью, построенной на этапе проектирования, были созданы шесть таблиц: Poezda, Rabotniki, Brigadi, Raspisanie, Sostav_Brigad и Prodazhi. Каждая таблица содержит ограничение первичного ключа (PRIMARY KEY), обеспечивающее уникальность записей, а межтабличные связи реализованы с помощью ограничений внешнего ключа (FOREIGN KEY), что гарантирует ссылочную целостность данных.

Первой была создана таблица Rabotniki, хранящая сведения о сотрудниках вокзала. Соответствующая SQL–команда представлена в соответствии с рисунком 9.

```
CREATE TABLE Rabotniki
(
    ID_Rabotnika      INT           NOT NULL,
    Familia           VARCHAR(50)   NOT NULL,
    Name              VARCHAR(50)   NOT NULL,
    Otchestvo         VARCHAR(50)   NULL,
    Data_rozhdenia    DATE          NOT NULL,
    Data_postuplenia  DATE          NULL,
    Stazh AS (CASE WHEN Data_postuplenia IS NOT NULL THEN DATEDIFF(YEAR, Data_postuplenia, GETDATE()) ELSE NULL END),
    Dolzhnost  VARCHAR(50)   NOT NULL,
    Gender     VARCHAR(10)   NOT NULL,
    Address    VARCHAR(100)  NULL,
    Gorod      VARCHAR(50)   NULL,
    Telefon    VARCHAR(20)   NOT NULL,
    CONSTRAINT PK_Rabotniki          PRIMARY KEY (ID_Rabotnika),
    CONSTRAINT UQ_Rabotniki_Telefon  UNIQUE (Telefon),
    CONSTRAINT CK_Rabotniki_Gender   CHECK (Gender IN ('М', 'Ж')),
    CONSTRAINT CK_Rabotniki_Dolzhnost CHECK (
        Dolzhnost IN ('Машинист', 'Проводник', 'Диспетчер', 'Кассир', 'Начальник поезда', 'Механик')),
    CONSTRAINT CK_Rabotniki_Dates    CHECK (Data_postuplenia > Data_rozhdenia));
```

Рисунок 9 – Создание таблицы Rabotniki

Таблица Rabotniki содержит столбцы: ID_Rabotnika (первичный ключ, тип INT), Familia, Name, Otchestvo, Gorod, Address (тип VARCHAR), Data_rozhdenia и Data_postuplenia (тип DATE), а также Dolzhnost, Gender и Telefon. На столбец Telefon наложено ограничение уникальности (UNIQUE), исключающее дублирование номеров телефонов. Ограничение CHECK на столбце Gender допускает только значения «М» или «Ж», а ограничение CHECK на столбце Dolzhnost ограничивает перечень должностей значениями:

«Машинист», «Проводник», «Диспетчер», «Кассир», «Начальник поезда», «Механик». Дополнительно реализовано ограничение, запрещающее дату поступления на работу раньше даты рождения сотрудника. В таблице также присутствует вычисляемый столбец Stazh, автоматически определяющий стаж работника в годах на основе функции DATEDIFF от Data_postuplenia до текущей даты.

Далее была создана связующая таблица Sostav_Brigad, реализующая связь «многие ко многим» между работниками и бригадами. Результат представлен в соответствии с рисунком 10.

```
CREATE TABLE Sostav_Brigad
(
    ID_Rabotnika  INT   NOT NULL,
    ID_brigadi    INT   NOT NULL,
    CONSTRAINT PK_Sostav_Brigad PRIMARY KEY (ID_Rabotnika, ID_brigadi),
    CONSTRAINT FK_Sostav_Rabotnik
        FOREIGN KEY (ID_Rabotnika) REFERENCES Rabotniki(ID_Rabotnika)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT FK_Sostav_Brigada
        FOREIGN KEY (ID_brigadi) REFERENCES Brigadi(ID_brigadi)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
);
```

Рисунок 10 – Создание таблицы Sostav_Brigad

Таблица Sostav_Brigad состоит из двух столбцов: ID_Rabotnika и ID_brigadi, каждый из которых является внешним ключом, ссылающимся на таблицы Rabotniki и Brigadi соответственно. Первичный ключ таблицы является составным и образован сочетанием обоих столбцов, что исключает дублирование одной и той же пары «работник – бригада». Подобная структура позволяет одному работнику числиться в нескольких бригадах, а одной бригаде включать неограниченное число сотрудников.

Следующей была создана таблица Brigadi, содержащая сведения о бригадах поездов. Результат представлен в соответствии с рисунком 11.

```sql
CREATE TABLE Brigadi
(
    ID_brigadi  INT   NOT NULL,
    ID_poezda   INT   NOT NULL,
    CONSTRAINT PK_Brigadi PRIMARY KEY (ID_brigadi),
    CONSTRAINT FK_Brigadi_Poezda
        FOREIGN KEY (ID_poezda) REFERENCES Poezda(ID_poezda)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Рисунок 11 – Создание таблицы Brigadi

Таблица Brigadi включает столбцы ID_brigadi (первичный ключ) и ID_poezda (внешний ключ, ссылающийся на таблицу Poezda). Для внешнего ключа FK_Brigadi_Poezda заданы параметры ON UPDATE CASCADE и ON DELETE CASCADE, что означает автоматическое обновление или удаление записей о бригадах при изменении или удалении соответствующего поезда в родительской таблице.

Затем была создана таблица Poezda, хранящая информацию о поездах. Результат представлен в соответствии с рисунком 12.

```sql
CREATE TABLE Poezda
(
    ID_poezda      INT           NOT NULL,
    Data_vipuska   DATE          NOT NULL,
    Kol_vagonov    INT           NULL,
    Tip_poezda     VARCHAR(50)   NOT NULL,
    CONSTRAINT PK_Poezda PRIMARY KEY (ID_poezda),
    CONSTRAINT CK_Poezda_Tip    CHECK (Tip_poezda IN ('Общий', 'Скоростной', 'Высокоскоростной')),
    CONSTRAINT CK_Poezda_Vagony CHECK (Kol_vagonov > 0)
);
```

Рисунок 12 – Создание таблицы Poezda

Таблица Poezda содержит столбцы: ID_poezda (первичный ключ), Data_vipuska (тип DATE), Kol_vagonov (тип INT) и Tip_poezda (тип VARCHAR). На столбец Tip_poezda наложено ограничение CHECK, допускающее только три значения: «Общий», «Скоростной» и «Высокоскоростной». На столбец Kol_vagonov также наложено ограничение CHECK, запрещающее нулевые и отрицательные значения количества вагонов.

После этого была создана таблица Raspisanie, описывающая расписание движения поездов. Результат представлен в соответствии с рисунком 13.

```
CREATE TABLE Raspisanie
(
    ID_reysa        INT             NOT NULL,
    ID_poezda       INT             NOT NULL,
    Data_otprav     DATE            NOT NULL,
    Otprav_time     TIME            NOT NULL,
    Mesto_otprav    VARCHAR(100)    NOT NULL,
    Data_prib       DATE            NOT NULL,
    Prib_time       TIME            NOT NULL,
    Mesto_prib      VARCHAR(100)    NOT NULL,
    CONSTRAINT PK_Raspisanie PRIMARY KEY (ID_reysa),
    CONSTRAINT FK_Raspisanie_Poezda
    FOREIGN KEY (ID_poezda) REFERENCES Poezda(ID_poezda) ON UPDATE CASCADE ON DELETE NO ACTION,
    CONSTRAINT CK_Raspisanie_Dates CHECK (Data_prib >= Data_otprav),
);
```

Рисунок 13 – Создание таблицы Raspisanie

Таблица Raspisanie включает столбцы: ID_reysa (первичный ключ), ID_poezda (внешний ключ на таблицу Poezda), Data_otprav и Data_prib (тип DATE), Otprav_time и Prib_time (тип TIME), Mesto_otprav, Mesto_prib, Ограничение CHECK на датах гарантирует, что дата прибытия не может быть раньше даты отправления.

Последней была создана таблица Prodazhi, регистрирующая факты продажи билетов. Результат представлен в соответствии с рисунком 14.

```
CREATE TABLE Prodazhi
(
    ID_bileta       INT             NOT NULL,
    Date_prodazhi   DATETIME        NOT NULL,
    FIO_passazhira  VARCHAR(150)    NOT NULL,
    Passport_info   VARCHAR(50)     NOT NULL,
    ID_reysa        INT             NOT NULL,
    Kol_billetov    INT             NOT NULL,
    Lgoti           VARCHAR(50)     NULL,
    Stoimost        DECIMAL(10,2)   NOT NULL,
    CONSTRAINT PK_Prodazhi          PRIMARY KEY (ID_bileta),
    CONSTRAINT FK_Prodazhi_Reys
        FOREIGN KEY (ID_reysa) REFERENCES Raspisanie(ID_reysa) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT CK_Prodazhi_Kol      CHECK (Kol_billetov > 0),
    CONSTRAINT CK_Prodazhi_Stoimost CHECK (Stoimost > 0),
    CONSTRAINT CK_Prodazhi_Lgoti    CHECK (
        Lgoti IN ('Пенсионер', 'Студент', 'Инвалид', 'Ветеран', 'Дети') OR Lgoti IS NULL));
```

Рисунок 14 – Создание таблицы Prodazhi

Таблица Prodazhi содержит столбцы: ID_bileta (первичный ключ), Date_prodazhi (тип DATETIME), FIO_passazhira и Passport_info (тип

VARCHAR), ID_reysa (внешний ключ на таблицу Raspisanie), Kol_billetov и Stoimost (тип DECIMAL), а также Lgoti (тип VARCHAR). Ограничение CHECK на столбце Lgoti допускает только значения из перечня: «Пенсионер», «Студент», «Инвалид», «Ветеран», «Дети», либо значение NULL при отсутствии льготы. Ограничения CHECK на столбцах Kol_billetov и Stoimost запрещают нулевые и отрицательные значения.

Для наглядного представления итоговой структуры базы данных и всех связей между таблицами в среде SSMS была построена диаграмма базы данных, представленная в соответствии с рисунком 15.

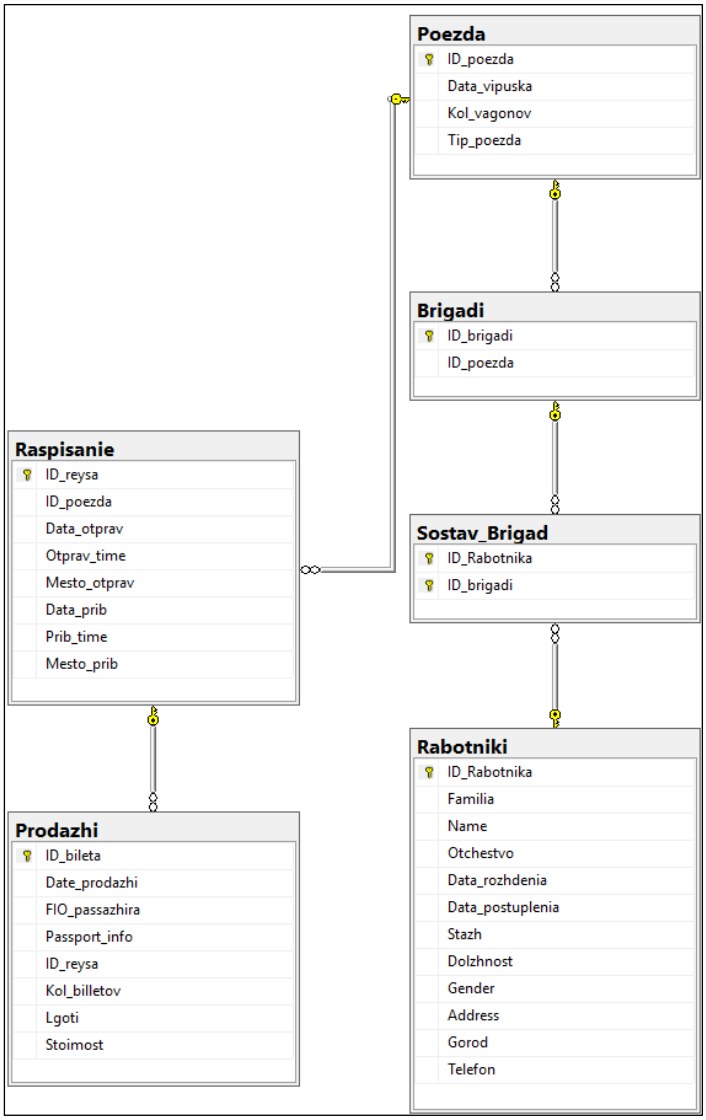

Рисунок 15 – Диаграмма базы данных ZheleznodorozhniyVokzal

Диаграмма отражает все разработанные таблицы, их атрибуты, а также связи между ними.

На диаграмме представлены следующие основные связи:

- связь «один ко многим» между таблицами Poezda и Brigadi;
- связь «один ко многим» между таблицами Poezda и Raspisanie;
- связь «один ко многим» между таблицами Raspisanie и Prodazhi;
- связь «многие ко многим», реализованная через таблицу Sostav_Brigad, между таблицами Rabotniki и Brigadi.

Использование диаграммы позволило визуально проверить корректность структуры базы данных и убедиться в правильности организации связей между сущностями.

После создания структуры базы данных каждая таблица была заполнена тестовыми данными с использованием среды Microsoft SQL Server Management Studio.

Процесс заполнения таблиц представлен в соответствии с рисунками 16–21.

```
INSERT INTO Rabotniki
    (ID_Rabotnika, Familia, Name, Otchestvo, Data_rozhdenia, Data_postuplenia, Dolzhnost, Gender, Address, Gorod, Telefon)
VALUES
(1,  'Иванов',    'Алексей',    'Петрович',     '1975-04-12', '2000-09-01', 'Машинист',         'М', 'ул. Ленина, 15',       'Москва',          '+79161234501'),
(2,  'Петрова',   'Мария',      'Ивановна',     '1982-07-23', '2005-06-15', 'Проводник',        'Ж', 'ул. Мира, 7',          'Санкт-Петербург', '+79162234502'),
(3,  'Сидоров',   'Дмитрий',    'Николаевич',   '1979-01-05', '2003-03-20', 'Диспетчер',        'М', 'пр. Победы, 33',       'Москва',          '+79163234503'),
(4,  'Козлова',   'Елена',      'Сергеевна',    '1990-11-17', '2012-08-01', 'Кассир',           'Ж', 'ул. Садовая, 21',      'Казань',          '+79164234504'),
(5,  'Смирнов',   'Игорь',      'Андреевич',    '1968-03-30', '1995-11-10', 'Начальник поезда', 'М', 'ул. Центральная, 8',   'Новосибирск',     '+79165234505'),
(6,  'Новикова',  'Анна',       'Викторовна',   '1985-09-14', '2008-04-25', 'Проводник',        'Ж', 'ул. Лесная, 44',       'Екатеринбург',    '+79166234506'),
(7,  'Морозов',   'Сергей',     'Александрович','1972-06-08', '1998-07-12', 'Машинист',         'М', 'ул. Рабочая, 56',      'Москва',          '+79167234507'),
(8,  'Волкова',   'Ольга',      'Михайловна',   '1988-12-22', '2010-02-14', 'Кассир',           'Ж', 'пр. Ленина, 12',       'Самара',          '+79168234508'),
(9,  'Соколов',   'Павел',      'Дмитриевич',   '1970-08-19', '1997-05-30', 'Машинист',         'М', 'ул. Советская, 3',     'Казань',          '+79169234509'),
(10, 'Михайлова', 'Татьяна',    'Юрьевна',      '1983-02-27', '2006-09-07', 'Проводник',        'Ж', 'ул. Школьная, 18',     'Москва',          '+79170234510'),
(11, 'Федоров',   'Николай',    'Владимирович', '1965-05-15', '1990-01-15', 'Диспетчер',        'М', 'пр. Мира, 29',         'Санкт-Петербург', '+79171234511'),
(12, 'Яковлева',  'Светлана',   'Олеговна',     '1993-10-04', '2015-11-20', 'Кассир',           'Ж', 'ул. Дружбы, 5',        'Новосибирск',     '+79172234512'),
(13, 'Попов',     'Андрей',     'Геннадьевич',  '1977-07-31', '2001-06-06', 'Машинист',         'М', 'ул. Пушкина, 67',      'Екатеринбург',    '+79173234513'),
(14, 'Лебедева',  'Наталья',    'Романовна',    '1986-04-16', '2009-03-01', 'Проводник',        'Ж', 'ул. Горького, 11',     'Самара',          '+79174234514'),
(15, 'Захаров',   'Виктор',     'Степанович',   '1960-11-28', '1985-08-15', 'Начальник поезда', 'М', 'пр. Победы, 77',       'Москва',          '+79175234515'),
(16, 'Горбунова', 'Ирина',      'Алексеевна',   '1991-03-09', '2013-07-10', 'Кассир',           'Ж', 'ул. Чехова, 34',       'Казань',          '+79176234516'),
(17, 'Кузнецов',  'Константин', 'Павлович',     '1973-09-21', '1999-10-25', 'Механик',          'М', 'ул. Строителей, 9',    'Санкт-Петербург', '+79177234517'),
(18, 'Орлова',    'Валентина',  'Федоровна',    '1980-01-13', '2004-04-18', 'Проводник',        'Ж', 'ул. Весенняя, 22',     'Новосибирск',     '+79178234518'),
(19, 'Тихонов',   'Роман',      'Борисович',    '1976-06-25', '2002-12-03', 'Диспетчер',        'М', 'пр. Гагарина, 48',     'Екатеринбург',    '+79179234519'),
(20, 'Зайцева',   'Людмила',    'Николаевна',   '1984-08-07', '2007-05-22', 'Кассир',           'Ж', 'ул. Молодежная, 6',    'Самара',          '+79180234520'),
(21, 'Белов',     'Евгений',    'Игоревич',     '1969-12-03', '1996-02-14', 'Машинист',         'М', 'ул. Первомайская, 14', 'Москва',          '+79181234521'),
(22, 'Крылова',   'Надежда',    'Вячеславовна', '1987-05-18', '2011-08-30', 'Проводник',        'Ж', 'ул. Заречная, 3',      'Казань',          '+79182234522'),
(23, 'Воробьев',  'Артем',      'Леонидович',   '1995-09-02', '2018-01-09', 'Механик',          'М', 'пр. Комсомольский, 25','Санкт-Петербург', '+79183234523'),
(24, 'Соловьева', 'Галина',     'Аркадьевна',   '1978-03-14', '2003-07-15', 'Диспетчер',        'Ж', 'ул. Кленовая, 17',     'Новосибирск',     '+79184234524'),
(25, 'Карпов',    'Леонид',     'Семенович',    '1963-07-29', '1988-06-20', 'Начальник поезда', 'М', 'ул. Парковая, 31',     'Екатеринбург',    '+79185234525');
```

Рисунок 16 – Заполнение таблицы Rabotniki

```
INSERT INTO Brigadi (ID_brigadi, ID_poezda) VALUES
(1,  1), (2,  1), (3,  2), (4,  2), (5,  3),
(6,  4), (7,  5), (8,  5), (9,  6), (10, 7),
(11, 7), (12, 8), (13, 9), (14, 10),(15, 10),
(16, 11),(17, 12),(18, 13),(19, 13),(20, 14),
(21, 15),(22, 16),(23, 17),(24, 18),(25, 19);
```

Рисунок 17– Заполнение таблицы Brigadi

```sql
INSERT INTO Poezda (ID_poezda, Data_vipuska, Kol_vagonov, Tip_poezda) VALUES
(1,  '2005-03-15', 16, 'Общий'),
(2,  '2010-07-22', 12, 'Скоростной'),
(3,  '2015-11-08',  8, 'Высокоскоростной'),
(4,  '2008-01-30', 18, 'Общий'),
(5,  '2012-05-17', 14, 'Скоростной'),
(6,  '2018-09-25', 10, 'Высокоскоростной'),
(7,  '2003-06-12', 20, 'Общий'),
(8,  '2016-02-28', 12, 'Скоростной'),
(9,  '2020-12-01',  6, 'Высокоскоростной'),
(10, '2007-08-14', 16, 'Общий'),
(11, '2011-04-03', 14, 'Скоростной'),
(12, '2019-10-19',  8, 'Высокоскоростной'),
(13, '2004-07-07', 22, 'Общий'),
(14, '2013-03-21', 12, 'Скоростной'),
(15, '2021-06-15',  6, 'Высокоскоростной'),
(16, '2006-11-11', 18, 'Общий'),
(17, '2014-08-09', 14, 'Скоростной'),
(18, '2022-01-30',  8, 'Высокоскоростной'),
(19, '2009-05-23', 20, 'Общий'),
(20, '2017-12-05', 10, 'Скоростной'),
(21, '2023-03-18',  6, 'Высокоскоростной'),
(22, '2002-09-27', 24, 'Общий'),
(23, '2015-07-14', 12, 'Скоростной'),
(24, '2020-04-22',  8, 'Высокоскоростной'),
(25, '2001-12-31', 16, 'Общий');
```

Рисунок 18– Заполнение таблицы Poezda

```sql
INSERT INTO Sostav_Brigad (ID_Rabotnika, ID_brigadi) VALUES
(1,  1), (2,  1),
(3,  1), (5,  2),
(10, 2), (7,  3),
(6,  3), (9,  4),
(11, 5), (13, 5),
(15, 6), (4,  6),
(21, 7), (14, 7),
(17, 8), (18, 8),
(1,  9), (22, 9),
(3,  10),(19, 10),
(8,  11),(24, 11),
(25, 12),
(12, 13),
(20, 14),
(2,  15);
```

Рисунок 19 – Заполнение таблицы Sostav_Brigad

```sql
INSERT INTO Raspisanie
    (ID_reysa, ID_poezda, Data_otprav, Otprav_time, Mesto_otprav, Data_prib, Prib_time, Mesto_prib)
VALUES
(1,  1,  '2025-01-10','08:00','Москва',           '2025-01-12','10:30','Владивосток'),
(2,  2,  '2025-01-11','09:15','Санкт-Петербург',  '2025-01-11','14:45','Москва'),
(3,  3,  '2025-01-12','07:00','Москва',            '2025-01-12','09:30','Санкт-Петербург'),
(4,  4,  '2025-01-13','14:00','Казань',            '2025-01-14','08:20','Москва'),
(5,  5,  '2025-01-14','11:30','Москва',            '2025-01-15','06:15','Новосибирск'),
(6,  6,  '2025-01-15','06:30','Новосибирск',       '2025-01-15','10:00','Омск'),
(7,  7,  '2025-01-16','22:00','Москва',            '2025-01-17','09:45','Екатеринбург'),
(8,  8,  '2025-01-17','10:00','Екатеринбург',      '2025-01-17','14:30','Челябинск'),
(9,  9,  '2025-01-18','07:45','Москва',            '2025-01-18','10:15','Нижний Новгород'),
(10, 10, '2025-01-19','15:00','Самара',            '2025-01-20','05:30','Москва'),
(11, 11, '2025-01-20','08:20','Москва',            '2025-01-21','20:00','Сочи'),
(12, 12, '2025-01-21','06:00','Сочи',              '2025-01-22','18:30','Москва'),
(13, 13, '2025-01-22','23:00','Москва',            '2025-01-24','06:00','Омск'),
(14, 14, '2025-01-23','12:00','Казань',            '2025-01-23','16:30','Самара'),
(15, 15, '2025-01-24','07:00','Москва',            '2025-01-24','09:00','Нижний Новгород'),
(16, 16, '2025-01-25','18:00','Новосибирск',       '2025-01-27','10:15','Москва'),
(17, 17, '2025-01-26','09:00','Санкт-Петербург',   '2025-01-26','14:00','Псков'),
(18, 18, '2025-01-27','05:30','Москва',            '2025-01-27','07:30','Тула'),
(19, 19, '2025-01-28','20:00','Ростов-на-Дону',    '2025-01-29','08:00','Москва'),
(20, 20, '2025-01-29','10:30','Москва',            '2025-01-30','22:00','Красноярск'),
(21, 21, '2025-01-30','06:00','Москва',            '2025-01-30','07:20','Тверь'),
(22, 22, '2025-01-31','14:00','Казань',            '2025-02-01','06:30','Новосибирск'),
(23, 23, '2025-02-01','09:45','Санкт-Петербург',   '2025-02-01','15:00','Вологда'),
(24, 24, '2025-02-02','07:00','Москва',            '2025-02-02','08:30','Рязань'),
(25, 25, '2025-02-03','22:30','Москва',            '2025-02-05','14:00','Владивосток');
```

Рисунок 20 – Заполнение таблицы Raspisanie

```sql
INSERT INTO Prodazhi
    (ID_bileta, Date_prodazhi, FIO_passazhira, Passport_info, ID_reysa, Kol_billetov, Lgoti, Stoimost)
VALUES
(1,  '20250105 10:15:00', 'Кириллов Иван Степанович',      '4512 123456', 1,  1, NULL,        8500.00),
(2,  '20250106 11:30:00', 'Носова Анастасия Олеговна',     '7823 654321', 2,  2, 'Студент',   2400.00),
(3,  '20250107 09:00:00', 'Дмитриев Кирилл Юрьевич',      '4517 789012', 3,  1, NULL,        3500.00),
(4,  '20250108 14:45:00', 'Романова Виктория Сергеевна',   '3312 345678', 4,  2, 'Пенсионер', 4200.00),
(5,  '20250108 16:00:00', 'Ершов Александр Борисович',     '5621 901234', 5,  1, NULL,        4200.00),
(6,  '20250109 08:20:00', 'Тимофеева Людмила Аркадьевна',  '6734 567890', 6,  3, 'Пенсионер', 8400.00),
(7,  '20250109 12:10:00', 'Власов Денис Николаевич',       '2245 123789', 7,  1, NULL,        3000.00),
(8,  '20250110 07:30:00', 'Гусева Марина Павловна',        '8956 432198', 8,  2, 'Студент',   1900.00),
(9,  '20250110 15:50:00', 'Кравцов Олег Витальевич',       '1123 876543', 9,  1, NULL,        4100.00),
(10, '20250111 10:00:00', 'Антонова Ксения Дмитриевна',    '3478 210987', 10, 2, 'Инвалид',   5400.00),
(11, '20250112 09:45:00', 'Мельников Юрий Александрович',  '5589 654012', 11, 1, NULL,        3800.00),
(12, '20250113 13:20:00', 'Степанова Екатерина Ивановна',  '7701 098765', 12, 2, NULL,        7800.00),
(13, '20250114 11:00:00', 'Никифоров Василий Петрович',    '9012 345123', 13, 1, 'Ветеран',   4500.00),
(14, '20250115 08:40:00', 'Осипова Вера Константиновна',   '2234 678901', 14, 3, 'Студент',   5400.00),
(15, '20250115 16:30:00', 'Панков Станислав Михайлович',   '4456 234567', 15, 1, NULL,        3200.00),
(16, '20250116 10:15:00', 'Рыбакова Нина Геннадьевна',     '6678 890123', 16, 2, 'Пенсионер',10000.00),
(17, '20250117 14:00:00', 'Семенов Григорий Олегович',     '8890 456789', 17, 1, NULL,         950.00),
(18, '20250118 07:55:00', 'Тарасова Ульяна Романовна',     '1102 012345', 18, 4, 'Дети',      2600.00),
(19, '20250119 12:30:00', 'Фролов Максим Андреевич',       '3314 568901', 19, 1, NULL,        3100.00),
(20, '20250120 09:00:00', 'Харитонова Алла Викторовна',    '5526 124567', 20, 2, NULL,       12400.00),
(21, '20250121 15:10:00', 'Цветков Артур Леонидович',      '7738 680123', 21, 1, NULL,        2100.00),
(22, '20250122 11:45:00', 'Чернова Диана Сергеевна',       '9950 236789', 22, 2, 'Студент',   9600.00),
(23, '20250123 08:00:00', 'Шарапова Полина Юрьевна',       '1162 792345', 23, 1, 'Инвалид',   1600.00),
(24, '20250124 13:25:00', 'Щербаков Роман Алексеевич',     '3374 348901', 24, 3, NULL,        5700.00),
(25, '20250125 10:50:00', 'Юрьева Наталья Борисовна',      '5586 904567', 25, 1, 'Ветеран',   9800.00);
```

Рисунок 21 – Зполнение таблицы Prodazhi

Таким образом, в рамках данного этапа была полностью разработана структура базы данных «Железнодорожный вокзал» в среде Microsoft SQL Server. На основе ранее построенной даталогической модели были созданы все необходимые таблицы, определены первичные и внешние ключи, а также реализованы ограничения целостности данных, обеспечивающие корректность и непротиворечивость хранимой информации.

4 Создание запросов

4.1 Создание запросов на выборку

Запросы являются основным инструментом работы с данными в реляционных базах данных. С помощью языка T–SQL в среде Microsoft SQL Server Management Studio были разработаны запросы трёх категорий: запросы на выборку, многотабличные запросы на соединение, а также запросы на группировку, изменение данных и использование встроенных функций. Все запросы выполнялись на базе данных «Железнодорожный вокзал».

Запросы на выборку предназначены для извлечения данных из таблиц базы данных с применением различных условий фильтрации. Основу составляет оператор SELECT в сочетании с фразой WHERE. В рамках данного подраздела были реализованы восемь запросов, каждый из которых демонстрирует отдельный механизм отбора данных.

Результаты выполнения запросов на выборку представлены в соответствии с рисунками 22–29.

```
SELECT ID_poezda as Идентификатор_поезда, Tip_poezda as тип_поезда,
Kol_vagonov as кол_вагонов, Data_vipuska as дата_выпуска
FROM Poezda
WHERE Kol_vagonov > 14;
```

110 %

Результаты | Сообщения

	Идентификатор_поезда	тип_поезда	кол_вагонов	дата_выпуска
1	1	Общий	16	2005-03-15
2	4	Общий	18	2008-01-30
3	7	Общий	20	2003-06-12
4	10	Общий	16	2007-08-14
5	13	Общий	22	2004-07-07
6	16	Общий	18	2006-11-11
7	19	Общий	20	2009-05-23
8	22	Общий	24	2002-09-27
9	25	Общий	16	2001-12-31

Рисунок 22 – Запрос с использованием операторов сравнения

В данном примере представлен запрос на выборку данных из таблицы работников с использованием логических операторов. Запрос позволяет отобрать сотрудников с должностями «Машинист» и «Диспетчер», при этом исключая работников, находящихся в городе Москва. Результат представлен в соответствии с рисунком 23.

```
SELECT ID_Rabotnika as идентификатор_работника, Familia as фамилия,
Name as имя, Dolzhnost as должность, Gorod as город
FROM Rabotniki
WHERE (Dolzhnost = 'Машинист' OR Dolzhnost = 'Диспетчер')
  AND NOT Gorod = 'Москва';
```

110 %

Результаты | Сообщения

	идентификатор_работника	фамилия	имя	должность	город
1	9	Соколов	Павел	Машинист	Казань
2	11	Федоров	Николай	Диспетчер	Санкт-Петербург
3	13	Попов	Андрей	Машинист	Екатеринбург
4	19	Тихонов	Роман	Диспетчер	Екатеринбург
5	24	Соловьева	Галина	Диспетчер	Новосибирск

Рисунок 23 – Запрос с использованием логических операторов AND, OR и NOT

В следующем примере показан запрос, который, осуществляет выборку данных о билетах, где пассажиры имеют льготы «Пенсионер» или «Ветеран», стоимость билета превышает 3000, а количество билетов больше одного. Результат представлен в соответствии с рисунком 24

```
SELECT ID_bileta as идентификатор_билета, FIO_passazhira ФИО_пассажира,
Lgoti as льготы, Kol_billetov as кол_билетов, Stoimost as стоимость
FROM Prodazhi
WHERE (Lgoti = 'Пенсионер' OR Lgoti = 'Ветеран')
  AND Stoimost > 3000
  AND Kol_billetov > 1;
```

110 %

Результаты | Сообщения

	идентификатор_билета	ФИО_пассажира	льготы	кол_билетов	стоимость
1	4	Романова Виктория Сергеевна	Пенсионер	2	4200.00
2	6	Тимофеева Людмила Аркадьевна	Пенсионер	3	8400.00
3	16	Рыбакова Нина Геннадьевна	Пенсионер	2	10000.00

Рисунок 24 – Запрос на использование комбинации логических операторов

```sql
SELECT
    p.ID_bileta as идентификатор_билета,
    p.FIO_passazhira as ФИО_пассажира,
    r.Mesto_otprav + ' - ' + r.Mesto_prib AS маршрут,
    p.Stoimost                            AS стоимость_за_1,
    p.Kol_billetov as кол_билетов,
    p.Stoimost * p.Kol_billetov           AS итоговая_сумма
FROM Prodazhi p
JOIN Raspisanie r ON p.ID_reysa = r.ID_reysa;
```

110 %

Результаты | Сообщения

	идентификатор_билета	ФИО_пассажира	маршрут	стоимость_за_1	кол_билетов	итоговая_сумма
1	1	Кириллов Иван Степанович	Москва - Владивосток	8500.00	1	8500.00
2	2	Носова Анастасия Олеговна	Санкт-Петербург - Москва	2400.00	2	4800.00
3	3	Дмитриев Кирилл Юрьевич	Москва - Санкт-Петербург	3500.00	1	3500.00
4	4	Романова Виктория Сергеевна	Казань - Москва	4200.00	2	8400.00
5	5	Ершов Александр Борисович	Москва - Новосибирск	4200.00	1	4200.00
6	6	Тимофеева Людмила Аркадьевна	Новосибирск - Омск	8400.00	3	25200.00
7	7	Власов Денис Николаевич	Москва - Екатеринбург	3000.00	1	3000.00
8	8	Гусева Марина Павловна	Екатеринбург - Челябинск	1900.00	2	3800.00
9	9	Кравцов Олег Витальевич	Москва - Нижний Новгород	4100.00	1	4100.00
10	10	Антонова Ксения Дмитриевна	Самара - Москва	5400.00	2	10800.00
11	11	Мельников Юрий Александрович	Москва - Сочи	3800.00	1	3800.00
12	12	Степанова Екатерина Ивановна	Сочи - Москва	7800.00	2	15600.00
13	13	Никифоров Василий Петрович	Москва - Омск	4500.00	1	4500.00
14	14	Осипова Вера Константиновна	Казань - Самара	5400.00	3	16200.00
15	15	Панков Станислав Михайлович	Москва - Нижний Новгород	3200.00	1	3200.00
16	16	Рыбакова Нина Геннадьевна	Новосибирск - Москва	10000.00	2	20000.00
17	17	Семенов Григорий Олегович	Санкт-Петербург - Псков	950.00	1	950.00
18	18	Тарасова Ульяна Романовна	Москва - Тула	2600.00	4	10400.00
19	19	Фролов Максим Андреевич	Ростов-на-Дону - Москва	3100.00	1	3100.00
20	20	Харитонова Алла Викторовна	Москва - Красноярск	12400.00	2	24800.00
21	21	Цветков Артур Леонидович	Москва - Тверь	2100.00	1	2100.00
22	22	Чернова Диана Сергеевна	Казань - Новосибирск	9600.00	2	19200.00
23	23	Шарапова Полина Юрьевна	Санкт-Петербург - Вологда	1600.00	1	1600.00
24	24	Щербаков Роман Алексеевич	Москва - Рязань	5700.00	3	17100.00
25	25	Юрьева Наталья Борисовна	Москва - Владивосток	9800.00	1	9800.00

Рисунок 25 – Запрос на использование выражений над столбцами

```sql
SELECT ID_Rabotnika as идентификатор_работника, Familia as фамилия, Name as имя, Dolzhnost as должность, Gorod as город
FROM Rabotniki
WHERE Dolzhnost IN ('Машинист', 'Начальник поезда', 'Механик');
```

110 %

Результаты | Сообщения

	идентификатор_работника	фамилия	имя	должность	город
1	1	Иванов	Алексей	Машинист	Москва
2	5	Смирнов	Игорь	Начальник поезда	Новосибирск
3	7	Морозов	Сергей	Машинист	Москва
4	9	Соколов	Павел	Машинист	Казань
5	13	Попов	Андрей	Машинист	Екатеринбург
6	15	Захаров	Виктор	Начальник поезда	Москва
7	17	Кузнецов	Константин	Механик	Санкт-Петербург
8	21	Белов	Евгений	Машинист	Москва
9	23	Воробьев	Артем	Механик	Санкт-Петербург
10	25	Карпов	Леонид	Начальник поезда	Екатеринбург

Рисунок 26 – Запрос с проверкой на принадлежность множеству

```sql
SELECT r.ID_reysa as идентификатор_рейса, r.Mesto_otprav as место_отправления, r.Mesto_prib as место_прибытия,
r.Data_otprav as дата_отправления, p.Stoimost as стоимость
FROM Raspisanie r
join Prodazhi p on r.ID_reysa = p.ID_reysa
WHERE Stoimost BETWEEN 2000 AND 5000;
```

110 %

Результаты | Сообщения

	идентификатор_рейса	место_отправления	место_прибытия	дата_отправления	стоимость
1	2	Санкт-Петербург	Москва	2025-01-11	2400.00
2	3	Москва	Санкт-Петербург	2025-01-12	3500.00
3	4	Казань	Москва	2025-01-13	4200.00
4	5	Москва	Новосибирск	2025-01-14	4200.00
5	7	Москва	Екатеринбург	2025-01-16	3000.00
6	9	Москва	Нижний Новгород	2025-01-18	4100.00
7	11	Москва	Сочи	2025-01-20	3800.00
8	13	Москва	Омск	2025-01-22	4500.00
9	15	Москва	Нижний Новгород	2025-01-24	3200.00
10	18	Москва	Тула	2025-01-27	2600.00
11	19	Ростов-на-Дону	Москва	2025-01-28	3100.00
12	21	Москва	Тверь	2025-01-30	2100.00

Рисунок 27 – Запрос с проверкой на принадлежность диапазону значений

```sql
SELECT ID_Rabotnika as идентификатор_работника, Familia as фамилия, Name as имя, Dolzhnost as должность, Telefon as телефон
FROM Rabotniki
WHERE Familia LIKE 'C%';
```

110 %

Результаты | Сообщения

	идентификатор_работника	фамилия	имя	должность	телефон
1	3	Сидоров	Дмитрий	Диспетчер	+79163234503
2	5	Смирнов	Игорь	Начальник поезда	+79165234505
3	9	Соколов	Павел	Машинист	+79169234509
4	24	Соловьева	Галина	Диспетчер	+79184234524

Рисунок 28 – Запрос с проверкой на соответствие шаблону

```sql
SELECT ID_bileta as идентификатор_билета, FIO_passazhira as ФИО_пассажира, Lgoti as льготы,
Passport_info as пасспортные_данные, ID_reysa as идентификатор_рейса, Stoimost as стоимость
FROM Prodazhi
WHERE Lgoti IS NULL;
```

110 %

Результаты | Сообщения

	идентификатор_билета	ФИО_пассажира	льготы	пасспортные_данные	идентификатор_рейса	стоимость
1	1	Кириллов Иван Степанович	NULL	4512 123456	1	8500.00
2	3	Дмитриев Кирилл Юрьевич	NULL	4517 789012	3	3500.00
3	5	Ершов Александр Борисович	NULL	5621 901234	5	4200.00
4	7	Власов Денис Николаевич	NULL	2245 123789	7	3000.00
5	9	Кравцов Олег Витальевич	NULL	1123 876543	9	4100.00
6	11	Мельников Юрий Александрович	NULL	5589 654012	11	3800.00
7	12	Степанова Екатерина Ивановна	NULL	7701 098765	12	7800.00
8	15	Панков Станислав Михайлович	NULL	4456 234567	15	3200.00
9	17	Семенов Григорий Олегович	NULL	8890 456789	17	950.00
10	19	Фролов Максим Андреевич	NULL	3314 568901	19	3100.00
11	20	Харитонова Алла Викторовна	NULL	5526 124567	20	12400.00
12	21	Цветков Артур Леонидович	NULL	7738 680123	21	2100.00
13	24	Щербаков Роман Алексеевич	NULL	3374 348901	24	5700.00

Рисунок 29 – Запрос с проверкой на неопределенное значение

4.2 Создание многотабличных запросов. Запросы на соединение

Многотабличные запросы позволяют объединять данные из двух и более таблиц в единую результирующую выборку. В рамках данного подраздела реализованы запросы с использованием различных видов соединений.

Декартово произведение формирует все возможные сочетания строк двух таблиц без условий связи и используется как базовая теоретическая модель соединения. Соединение по равенству через фразу WHERE отбирает только те строки, у которых значения ключевых столбцов совпадают, а добавление дополнительных условий отбора позволяет сузить результат до нужного подмножества. Запрос с соединением трёх таблиц демонстрирует последовательное связывание Prodazhi, Raspisanie и Poezda для получения сводной информации о продажах. Аналогичные запросы были переработаны с использованием явного оператора INNER JOIN. Левое внешнее соединение (LEFT JOIN) включает в результат все строки левой таблицы, в том числе те, для которых нет соответствующих записей в правой. Правое внешнее соединение (RIGHT JOIN) работает симметрично. Запрос с симметричным соединением таблицы с самой собой позволяет выявить пары записей, удовлетворяющих заданному условию. Результаты выполнения многотабличных запросов представлены в соответствии с рисунками 30–39.

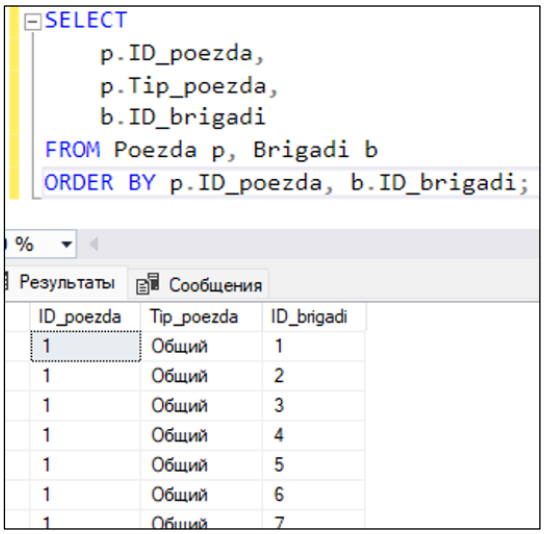

Рисунок 30 – Запрос с использованием декартового произведения двух таблиц

Следующий запрос соединяет таблицы Brigadi и Poezda по условию равенства внешнего и первичного ключей. В результирующей выборке для каждой бригады отображаются: номер бригады (ID_brigadi), номер поезда (ID_poezda), тип поезда (Tip_poezda) и количество вагонов (Kol_vagonov). Результат представлен в соответствии с рисунком 31.

```
SELECT
    b.ID_brigadi as идентификатор_бригады,
    p.ID_poezda as идентификатор_поезда,
    p.Tip_poezda as тип_поезда,
    p.Kol_vagonov as кол_вагонов
FROM Brigadi b, Poezda p
WHERE b.ID_poezda = p.ID_poezda;
```

110 %

Результаты | Сообщения

	идентификатор_бригады	идентификатор_поезда	тип_поезда	кол_вагонов
1	1	1	Общий	16
2	2	1	Общий	16
3	3	2	Скоростной	12
4	4	2	Скоростной	12
5	5	3	Высокоскоростной	8
6	6	4	Общий	18
7	7	5	Скоростной	14
8	8	5	Скоростной	14
9	9	6	Высокоскоростной	10
10	10	7	Общий	20
11	11	7	Общий	20
12	12	8	Скоростной	12
13	13	9	Высокоскоростной	6
14	14	10	Общий	16
15	15	10	Общий	16
16	16	11	Скоростной	14
17	17	12	Высокоскоростной	8
18	18	13	Общий	22
19	19	13	Общий	22
20	20	14	Скоростной	12
21	21	15	Высокоскоростной	6
22	22	16	Общий	18
23	23	17	Скоростной	14
24	24	18	Высокоскоростной	8
25	25	19	Общий	20

Рисунок 31 – Запрос с использованием соединения двух таблиц по равенству

```sql
SELECT
    r.ID_reysa as идентификатор_рейса,
    p.ID_poezda as идентификатор_поезда,
    p.Tip_poezda as тип_поезда,
    r.Mesto_otprav as место_отправления,
    r.Mesto_prib as место_прибытия,
    r.Data_otprav as дата_отправления
FROM Raspisanie r, Poezda p
WHERE r.ID_poezda = p.ID_poezda
  AND p.Tip_poezda = 'Скоростной'
  AND r.Mesto_otprav = 'Москва';
```

110 %

Результаты | Сообщения

	идентификатор_рейса	идентификатор_поезда	тип_поезда	место_отправления	место_прибытия	дата_отправления
1	5	5	Скоростной	Москва	Новосибирск	2025-01-14
2	11	11	Скоростной	Москва	Сочи	2025-01-20
3	20	20	Скоростной	Москва	Красноярск	2025-01-29

Рисунок 32 – Запрос с использованием соединения двух таблиц по равенству и условием отбора

```sql
SELECT
    pr.ID_bileta as идентификатор_билета,
    pr.FIO_passazhira as ФИО_пассажира,
    pr.Lgoti as льготы,
    r.Mesto_otprav as место_отправления,
    r.Mesto_prib as место_прибытия,
    r.Data_otprav as дата_отправления,
    p.Tip_poezda as тип_поезда,
    pr.Stoimost as стоимость
FROM Prodazhi pr, Raspisanie r, Poezda p
WHERE pr.ID_reysa = r.ID_reysa
  AND r.ID_poezda = p.ID_poezda;
```

110 %

Результаты | Сообщения

	идентификатор_билета	ФИО_пассажира	льготы	место_отправления	место_прибытия	дата_отправления	тип_поезда	стоимость
1	1	Кириллов Иван Степанович	NULL	Москва	Владивосток	2025-01-10	Общий	8500.00
2	2	Носова Анастасия Олеговна	Студент	Санкт-Петербург	Москва	2025-01-11	Скоростной	2400.00
3	3	Дмитриев Кирилл Юрьевич	NULL	Москва	Санкт-Петербург	2025-01-12	Высокоскоростной	3500.00
4	4	Романова Виктория Сергеевна	Пенсионер	Казань	Москва	2025-01-13	Общий	4200.00
5	5	Ершов Александр Борисович	NULL	Москва	Новосибирск	2025-01-14	Скоростной	4200.00
6	6	Тимофеева Людмила Аркадьевна	Пенсионер	Новосибирск	Омск	2025-01-15	Высокоскоростной	8400.00
7	7	Власов Денис Николаевич	NULL	Москва	Екатеринбург	2025-01-16	Общий	3000.00
8	8	Гусева Марина Павловна	Студент	Екатеринбург	Челябинск	2025-01-17	Скоростной	1900.00
9	9	Кравцов Олег Витальевич	NULL	Москва	Нижний Новгород	2025-01-18	Высокоскоростной	4100.00
10	10	Антонова Ксения Дмитриевна	Инвалид	Самара	Москва	2025-01-19	Общий	5400.00
11	11	Мельников Юрий Александрович	NULL	Москва	Сочи	2025-01-20	Скоростной	3800.00
12	12	Степанова Екатерина Ивановна	NULL	Сочи	Москва	2025-01-21	Высокоскоростной	7800.00
13	13	Никифоров Василий Петрович	Ветеран	Москва	Омск	2025-01-22	Общий	4500.00
14	14	Осипова Вера Константиновна	Студент	Казань	Самара	2025-01-23	Скоростной	5400.00
15	15	Панков Станислав Михайлович	NULL	Москва	Нижний Новгород	2025-01-24	Высокоскоростной	3200.00
16	16	Рыбакова Нина Геннадьевна	Пенсионер	Новосибирск	Москва	2025-01-25	Общий	10000.00
17	17	Семенов Григорий Олегович	NULL	Санкт-Петербург	Псков	2025-01-26	Скоростной	950.00
18	18	Тарасова Ульяна Романовна	Дети	Москва	Тула	2025-01-27	Высокоскоростной	2600.00
19	19	Фролов Максим Андреевич	NULL	Ростов-на-Дону	Москва	2025-01-28	Общий	3100.00
20	20	Харитонова Алла Викторовна	NULL	Москва	Красноярск	2025-01-29	Скоростной	12400.00
21	21	Цветков Артур Леонидович	NULL	Москва	Тверь	2025-01-30	Высокоскоростной	2100.00
22	22	Чернова Диана Сергеевна	Студент	Казань	Новосибирск	2025-01-31	Общий	9600.00
23	23	Шарапова Полина Юрьевна	Инвалид	Санкт-Петербург	Вологда	2025-02-01	Скоростной	1600.00
24	24	Щербаков Роман Алексеевич	NULL	Москва	Рязань	2025-02-02	Высокоскоростной	5700.00
25	25	Юрьева Наталья Борисовна	Ветеран	Москва	Владивосток	2025-02-03	Общий	9800.00

Рисунок 33 Запрос с использованием соединения по трем таблицам

```sql
SELECT
    b.ID_brigadi as идентификатор_бригады,
    p.ID_poezda as идентификатор_поезда,
    p.Tip_poezda as тип_поезда,
    p.Kol_vagonov as кол_вагонов
FROM Brigadi b
FULL JOIN Poezda p ON b.ID_poezda = p.ID_poezda;
```

110 %

Результаты | Сообщения

	идентификатор_бригады	идентификатор_поезда	тип_поезда	кол_вагонов
1	1	1	Общий	16
2	2	1	Общий	16
3	3	2	Скоростной	12
4	4	2	Скоростной	12
5	5	3	Высокоскоростной	8
6	6	4	Общий	18
7	7	5	Скоростной	14
8	8	5	Скоростной	14
9	9	6	Высокоскоростной	10
10	10	7	Общий	20
11	11	7	Общий	20
12	12	8	Скоростной	12
13	13	9	Высокоскоростной	6
14	14	10	Общий	16
15	15	10	Общий	16
16	16	11	Скоростной	14
17	17	12	Высокоскоростной	8
18	18	13	Общий	22
19	19	13	Общий	22
20	20	14	Скоростной	12
21	21	15	Высокоскоростной	6
22	22	16	Общий	18
23	23	17	Скоростной	14
24	24	18	Высокоскоростной	8
25	25	19	Общий	20
26	NULL	20	Скоростной	10
27	NULL	21	Высокоскоростной	6
28	NULL	22	Общий	24
29	NULL	23	Скоростной	12
30	NULL	24	Высокоскоростной	8
31	NULL	25	Общий	16

Рисунок 34 – Копия ранее созданного запроса на соединение по равенству с использованием внешнего полного соединения таблиц (JOIN).

```sql
SELECT
    r.ID_reysa as идентификатор_рейса,
    p.ID_poezda as идентификатор_поезда,
    p.Tip_poezda as тип_поезда,
    r.Mesto_otprav as место_отправления,
    r.Mesto_prib as место_прибытия,
    r.Data_otprav as дата_отправления
FROM Raspisanie r
FULL JOIN Poezda p ON r.ID_poezda = p.ID_poezda
WHERE p.Tip_poezda = 'Скоростной'
  AND r.Mesto_otprav = 'Москва';
```

110 %

Результаты | Сообщения

	идентификатор_рейса	идентификатор_поезда	тип_поезда	место_отправления	место_прибытия	дата_отправления
1	5	5	Скоростной	Москва	Новосибирск	2025-01-14
2	11	11	Скоростной	Москва	Сочи	2025-01-20
3	20	20	Скоростной	Москва	Красноярск	2025-01-29

Рисунок 35 Копия ранее созданного запроса на соединение по равенству с использованием внешнего полного соединения таблиц (JOIN).

```sql
SELECT
    r.Name,
    r.Familia,
    r.Otchestvo,
    r.ID_Rabotnika,
    s.ID_Brigadi
FROM Rabotniki r
left JOIN Sostav_Brigad s ON r.ID_Rabotnika = s.ID_Rabotnik
ORDER BY s.ID_brigadi;
```

100 %

Результаты | Сообщения

	Name	Familia	Otchestvo	ID_Rabotnika	ID_Brigadi
1	Ирина	Горбунова	Алексеевна	16	NULL
2	Артем	Воробьев	Леонидович	23	NULL
3	Алексей	Иванов	Петрович	1	1
4	Мария	Петрова	Ивановна	2	1
5	Дмитрий	Сидоров	Николаевич	3	1
6	Татьяна	Михайлова	Юрьевна	10	2
7	Игорь	Смирнов	Андреевич	5	2
8	Анна	Новикова	Викторовна	6	3
9	Сергей	Морозов	Александрович	7	3
10	Павел	Соколов	Дмитриевич	9	4
11	Николай	Федоров	Владимирович	11	5
12	Андрей	Попов	Геннадьевич	13	5
13	Елена	Козлова	Сергеевна	4	6
14	Виктор	Захаров	Степанович	15	6
15	Евгений	Белов	Игоревич	21	7
16	Наталья	Лебедева	Романовна	14	7

Рисунок 36 – Запрос с использованием левого внешнего соединения

```sql
SELECT
    r.Name as Имя_работника,
    r.Familia as фамилия,
    r.Otchestvo as отчество,
    r.ID_Rabotnika as идент_работника,
    p.ID_Poezda as идент_поезда,
    p.Tip_Poezda as тип_поезда,
    p.Kol_vagonov as кол_вагонов,
    b.ID_Brigadi as идент_бригады

FROM Poezda p
FULL JOIN Brigadi b ON p.ID_Poezda = b.ID_Poezda
FULL JOIN Sostav_Brigad s ON s.ID_Brigadi = b.ID_Brigadi
FULL JOIN Rabotniki r ON r.ID_Rabotnika = s.ID_Rabotnika
ORDER BY b.ID_Brigadi;
```

100 %

Результаты | Сообщения

	Имя_работника	фамилия	отчество	идент_работника	идент_поезда	тип_поезда	кол_вагонов	идент_бригады
1	Артем	Воробьев	Леонидович	23	NULL	NULL	NULL	NULL
2	Ирина	Горбунова	Алексеевна	16	NULL	NULL	NULL	NULL
3	NULL	NULL	NULL	NULL	23	Скоростной	12	NULL
4	NULL	NULL	NULL	NULL	24	Высокоскоростной	8	NULL
5	NULL	NULL	NULL	NULL	25	Общий	16	NULL
6	NULL	NULL	NULL	NULL	20	Скоростной	10	NULL
7	NULL	NULL	NULL	NULL	21	Высокоскоростной	6	NULL
8	NULL	NULL	NULL	NULL	22	Общий	24	NULL
9	Алексей	Иванов	Петрович	1	1	Общий	16	1
10	Мария	Петрова	Ивановна	2	1	Общий	16	1
11	Дмитрий	Сидоров	Николаевич	3	1	Общий	16	1
12	Татьяна	Михайлова	Юрьевна	10	1	Общий	16	2
13	Игорь	Смирнов	Андреевич	5	1	Общий	16	2
14	Анна	Новикова	Викторовна	6	2	Скоростной	12	3
15	Сергей	Морозов	Александрович	7	2	Скоростной	12	3
16	Павел	Соколов	Дмитриевич	9	2	Скоростной	12	4
17	Николай	Федоров	Владимирович	11	3	Высокоскоростной	8	5
18	Андрей	Попов	Геннадьевич	13	3	Высокоскоростной	8	5
19	Елена	Козлова	Сергеевна	4	4	Общий	18	6
20	Виктор	Захаров	Степанович	15	4	Общий	18	6
21	Евгений	Белов	Игоревич	21	5	Скоростной	14	7
22	Наталья	Лебедева	Романовна	14	5	Скоростной	14	7
23	Константин	Кузнецов	Павлович	17	5	Скоростной	14	8
24	Валентина	Орлова	Федоровна	18	5	Скоростной	14	8
25	Алексей	Иванов	Петрович	1	6	Высокоскоростной	10	9
26	Надежда	Крылова	Вячеславовна	22	6	Высокоскоростной	10	9
27	Дмитрий	Сидоров	Николаевич	3	7	Общий	20	10
28	Роман	Тихонов	Борисович	19	7	Общий	20	10
29	Галина	Соловьева	Аркадьевна	24	7	Общий	20	11
30	Ольга	Волкова	Михайловна	8	7	Общий	20	11

Рисунок 37 – Запрос на соединение по равенству с использованием внешнего полного соединения таблиц

Следующий запрос представляет результат правого внешнего соединения таблиц Sostav_Brigad и Rabotniki. Использование оператора RIGHT JOIN обеспечивает включение в выборку всех строк из правой таблицы

независимо от наличия соответствующих записей в связанных таблицах. Результат представлен в соответствии с рисунком 38

```
SELECT
    b.ID_brigadi идент_бригады,
    p.ID_poezda идент_поезда,
    p.Tip_poezda as тип_поезда,
    p.Kol_vagonov as кол_вагонов
FROM Brigadi b
RIGHT JOIN Poezda p ON b.ID_poezda = p.ID_poezda
ORDER BY p.ID_poezda desc;
```

100 %

Результаты | Сообщения

	идент_бригады	идент_поезда	тип_поезда	кол_вагонов
1	NULL	25	Общий	16
2	NULL	24	Высокоскоростной	8
3	NULL	23	Скоростной	12
4	NULL	22	Общий	24
5	NULL	21	Высокоскоростной	6
6	NULL	20	Скоростной	10
7	25	19	Общий	20
8	24	18	Высокоскоростной	8
9	23	17	Скоростной	14
10	22	16	Общий	18
11	21	15	Высокоскоростной	6
12	20	14	Скоростной	12
13	18	13	Общий	22
14	19	13	Общий	22
15	17	12	Высокоскоростной	8
16	16	11	Скоростной	14
17	14	10	Общий	16
18	15	10	Общий	16
19	13	9	Высокоскоростной	6
20	12	8	Скоростной	12
21	10	7	Общий	20
22	11	7	Общий	20
23	9	6	Высокоскоростной	10
24	7	5	Скоростной	14
25	8	5	Скоростной	14
26	6	4	Общий	18
27	5	3	Высокоскоростной	8
28	3	2	Скоростной	12
29	4	2	Скоростной	12
30	1	1	Общий	16
31	2	1	Общий	16

Рисунок 38– Запрос на использование правого внешнего соединения

Далее запрос представляет результат симметричного соединения таблицы Rabotniki с самой собой. Таблица используется дважды под псевдонимами r1 и

r2, а условие соединения r1.Gorod = r2.Gorod отбирает пары работников, проживающих в одном городе. Условие r1.ID_Rabotnika < r2.ID_Rabotnika исключает дублирующиеся пары и записи вида «работник с самим собой», обеспечивая уникальность каждой пары в результирующей выборке. Результат отсортирован по городу, что позволяет наглядно сгруппировать коллег по месту проживания. Результат показан в соответствии с рисунком 39

```
SELECT
    r1.ID_Rabotnika  AS идент_1_работника,
    r1.Familia       AS фаимлия_1,
    r2.ID_Rabotnika  AS идент_2_работника,
    r2.Familia       AS фамилия_2,
    r1.Gorod         AS город
FROM Rabotniki r1
JOIN Rabotniki r2
    ON r1.Gorod = r2.Gorod
   AND r1.ID_Rabotnika < r2.ID_Rabotnika
```

110 %

Результаты | Сообщения

	идент_1_работника	фаимлия_1	идент_2_работника	фамилия_2	город
1	6	Новикова	13	Попов	Екатеринбург
2	6	Новикова	19	Тихонов	Екатеринбург
3	6	Новикова	25	Карпов	Екатеринбург
4	13	Попов	19	Тихонов	Екатеринбург
5	13	Попов	25	Карпов	Екатеринбург
6	19	Тихонов	25	Карпов	Екатеринбург
7	16	Горбунова	22	Крылова	Казань
8	9	Соколов	16	Горбунова	Казань
9	9	Соколов	22	Крылова	Казань
10	4	Козлова	9	Соколов	Казань
11	4	Козлова	16	Горбунова	Казань
12	4	Козлова	22	Крылова	Казань
13	1	Иванов	3	Сидоров	Москва
14	1	Иванов	7	Морозов	Москва
15	1	Иванов	10	Михайлова	Москва
16	1	Иванов	15	Захаров	Москва
17	1	Иванов	21	Белов	Москва

Рисунок 39 – Запрос с использованием симметричного соединения и удаление избыточности

4.3 Создание запросов на группировку и сортировку данных. Запросы на изменение. Использование встроенных функций.

В данном подразделе реализованы запросы, использующие агрегатные и встроенные функции SQL Server, механизм группировки и сортировки, а также операторы изменения данных.

Результаты выполнения всех запросов данного подраздела представлены в соответствии с рисунками 40–49.

```
SELECT Dolzhnost as должность, COUNT(*) AS количество_работников
FROM Rabotniki
GROUP BY Dolzhnost;
```

110 %

Результаты | Сообщения

	должность	количество_работников
1	Диспетчер	4
2	Кассир	5
3	Машинист	5
4	Механик	2
5	Начальник поезда	3
6	Проводник	6

Рисунок 40 – Запрос с использованием функции COUNT

```
SELECT
    ISNULL(Lgoti, 'Без льготы') AS категория_льгот,
    SUM(kol_billetov)           AS количество_билетов,
    SUM(Stoimost)               AS общая_сумма
FROM Prodazhi
GROUP BY Lgoti
ORDER BY общая_сумма DESC;
```

110 %

Результаты | Сообщения

	категория_льгот	количество_билетов	общая_сумма
1	Без льготы	17	62350.00
2	Пенсионер	7	22600.00
3	Студент	9	19300.00
4	Ветеран	2	14300.00
5	Инвалид	3	7000.00
6	Дети	4	2600.00

Рисунок 41 – Запрос с использованием функции SUM

Представленный запрос формирует выборку данных о работниках, включая расчёт их возраста и стажа работы на текущую дату. Оператор CAST(GETDATE() AS DATE) обеспечивает приведение системной даты и времени к типу DATE, исключая временную составляющую (часы, минуты, секунды). В результате в столбце текущая дата отображается только год, месяц и день, что повышает наглядность отчёта и исключает избыточную информацию о времени. Результат показан в соответствии с рисунком 42

```
SELECT
    ID_Rabotnika AS идентификатор_работника,
    Familia AS фамилия,
    Name AS имя,
    CAST(Data_rozhdenia AS DATE) AS дата_рождения,
    DATEDIFF(YEAR, Data_rozhdenia, GETDATE()) -
    IIF(FORMAT(Data_rozhdenia, 'MM-dd') > FORMAT(GETDATE(), 'MM-dd'), 1, 0) AS возраст,
    CAST(Data_postuplenia AS DATE) AS дата_поступления,
    DATEDIFF(YEAR, Data_postuplenia, GETDATE()) -
    IIF(FORMAT(Data_postuplenia, 'MM-dd') > FORMAT(GETDATE(), 'MM-dd'), 1, 0) AS стаж,
    CAST(GETDATE() AS DATE) AS [текущая дата]
FROM Rabotniki
WHERE Data_postuplenia IS NOT NULL;
```

100 %

Результаты | Сообщения

	идентификатор_работника	фамилия	имя	дата_рождения	возраст	дата_поступления	стаж	текущая дата
1	1	Иванов	Алексей	1975-04-12	50	2000-09-01	25	2026-04-08
2	2	Петрова	Мария	1982-07-23	43	2005-06-15	20	2026-04-08
3	3	Сидоров	Дмитрий	1979-01-05	47	2003-03-20	23	2026-04-08
4	4	Козлова	Елена	1990-11-17	35	2012-08-01	13	2026-04-08
5	5	Смирнов	Игорь	1968-03-30	58	1995-11-10	30	2026-04-08
6	6	Новикова	Анна	1985-09-14	40	2008-04-25	17	2026-04-08
7	7	Морозов	Сергей	1972-06-08	53	1998-07-12	27	2026-04-08
8	8	Волкова	Ольга	1988-12-22	37	2010-02-14	16	2026-04-08
9	9	Соколов	Павел	1970-08-19	55	1997-05-30	28	2026-04-08
10	10	Михайлова	Татьяна	1983-02-27	43	2006-09-07	19	2026-04-08
11	11	Федоров	Николай	1965-05-15	60	1990-01-15	36	2026-04-08
12	12	Яковлева	Светлана	1993-10-04	32	2015-11-20	10	2026-04-08
13	13	Попов	Андрей	1977-07-31	48	2001-06-06	24	2026-04-08
14	14	Лебедева	Наталья	1986-04-16	39	2009-03-01	17	2026-04-08
15	15	Захаров	Виктор	1960-11-28	65	1985-08-15	40	2026-04-08
16	16	Горбунова	Ирина	1991-03-09	35	2013-07-10	12	2026-04-08
17	17	Кузнецов	Константин	1973-09-21	52	1999-10-25	26	2026-04-08
18	18	Орлова	Валентина	1980-01-13	46	2004-04-18	21	2026-04-08
19	19	Тихонов	Роман	1976-06-25	49	2002-12-03	23	2026-04-08
20	20	Зайцева	Людмила	1984-08-07	41	2007-05-22	18	2026-04-08
21	21	Белов	Евгений	1969-12-03	56	1996-02-14	30	2026-04-08
22	22	Крылова	Надежда	1987-05-18	38	2011-08-30	14	2026-04-08
23	23	Воробьев	Артем	1995-09-02	30	2018-01-09	8	2026-04-08
24	24	Соловьева	Галина	1978-03-14	48	2003-07-15	22	2026-04-08
25	25	Карпов	Леонид	1963-07-29	62	1988-06-20	37	2026-04-08

Рисунок 42 – Запрос с использованием временных функций

Представленный запрос демонстрирует использование группировки данных по одному столбцу с помощью оператора GROUP BY. Запрос группирует записи в таблице Poezda по типу поезда (Tip_poezda) и подсчитывает количество поездов в каждой группе с помощью агрегатной функции COUNT(*). Результат представлен в соответствии с рисунком 43

```
SELECT
    Tip_poezda,
    COUNT(*) AS Kolichestvo_poezdov
FROM Poezda
GROUP BY Tip_poezda;
```

110 %

Результаты | Сообщения

	Tip_poezda	Kolichestvo_poezdov
1	Высокоскоростной	8
2	Общий	9
3	Скоростной	8

Рисунок 43 – Запрос с использованием группировки по одному столбцу

Представленный запрос расширяет предыдущий пример добавлением условия отбора групп с помощью оператора HAVING. В отличие от WHERE, который фильтрует отдельные строки до группировки, HAVING применяется уже после формирования групп и позволяет отсеять те группы, которые не удовлетворяют заданному условию.

```
SELECT
    Tip_poezda,
    COUNT(*) AS Kolichestvo_poezdov
FROM Poezda
GROUP BY Tip_poezda
HAVING COUNT(*) > 8;
```

110 %

Результаты | Сообщения

	Tip_poezda	Kolichestvo_poezdov
1	Общий	9

Рисунок 44 – Запрос с использованием условия отбора групп HAVING

```sql
SELECT
    Raspisanie.ID_reysa as идентификатор_рейса,
    Mesto_otprav as место_отправления,
    Mesto_prib as место_прибытия,
    Data_otprav as дата_отправления,
    Stoimost as стоимсоть
FROM Raspisanie, Prodazhi
WHERE Raspisanie.ID_reysa = Prodazhi.ID_reysa
ORDER BY Stoimost DESC;
```

110 %

Результаты | Сообщения

	идентификатор_рейса	место_отправления	место_прибытия	дата_отправления	стоимсоть
1	20	Москва	Красноярск	2025-01-29	12400.00
2	16	Новосибирск	Москва	2025-01-25	10000.00
3	25	Москва	Владивосток	2025-02-03	9800.00
4	22	Казань	Новосибирск	2025-01-31	9600.00
5	1	Москва	Владивосток	2025-01-10	8500.00
6	6	Новосибирск	Омск	2025-01-15	8400.00
7	12	Сочи	Москва	2025-01-21	7800.00
8	24	Москва	Рязань	2025-02-02	5700.00
9	10	Самара	Москва	2025-01-19	5400.00
10	14	Казань	Самара	2025-01-23	5400.00
11	13	Москва	Омск	2025-01-22	4500.00
12	4	Казань	Москва	2025-01-13	4200.00
13	5	Москва	Новосибирск	2025-01-14	4200.00
14	9	Москва	Нижний Новгород	2025-01-18	4100.00
15	11	Москва	Сочи	2025-01-20	3800.00
16	3	Москва	Санкт-Петербург	2025-01-12	3500.00
17	15	Москва	Нижний Новгород	2025-01-24	3200.00
18	19	Ростов-на-Дону	Москва	2025-01-28	3100.00
19	7	Москва	Екатеринбург	2025-01-16	3000.00
20	18	Москва	Тула	2025-01-27	2600.00
21	2	Санкт-Петербург	Москва	2025-01-11	2400.00
22	21	Москва	Тверь	2025-01-30	2100.00
23	8	Екатеринбург	Челябинск	2025-01-17	1900.00
24	23	Санкт-Петербург	Вологда	2025-02-01	1600.00
25	17	Санкт-Петербург	Псков	2025-01-26	950.00

Рисунок 45 – Запрос с использованием сортировки по столбцу

```sql
INSERT INTO Poezda (Data_vipuska, Kol_vagonov, Tip_poezda)
VALUES ('2024-05-10', 10, 'Скоростной');
```

110 %

Сообщения

```
(затронута одна строка)
```

Рисунок 46 – Запрос на добавление новых данных в таблицу

```
UPDATE Poezda
SET Kol_vagonov = 12
WHERE ID_poezda = 26;
```

110 %

Сообщения

```
(затронута одна строка)
```

Рисунок 47 – Запрос на обновление существующих данных в таблице

```
UPDATE Prodazhi
SET Stoimost = Stoimost * 1.05
WHERE ID_reysa IN (
    SELECT ID_reysa
    FROM Raspisanie
    WHERE Data_otprav > '20250120'
);
```

110 %

Сообщения

```
(затронуто строк: 14)
```

Рисунок 48 – Запрос на обновление существующих данных по результатам подзапроса во фразе WHERE

```
DELETE FROM Poezda
WHERE ID_poezda = 26;
```

10 %

Сообщения

```
(затронута одна строка)
```

Рисунок 49 – Запрос на удаление существующих данных

Таким образом, в рамках раздела 4 были реализованы все предусмотренные типы запросов к базе данных «Железнодорожный вокзал». Разработанные запросы охватывают основные механизмы языка T–SQL и позволяют решать практические задачи выборки, анализа и изменения данных в условиях реальной эксплуатации системы.

5 Создание представлений

Создание представлений является важным этапом разработки базы данных, так как позволяет упростить доступ к данным, повысить удобство работы пользователей и обеспечить дополнительный уровень безопасности.

Представление представляет собой виртуальную таблицу, формируемую на основе SQL–запроса. В отличие от обычных таблиц, представления не хранят данные физически, а отображают результат выполнения запроса к одной или нескольким таблицам базы данных.

В рамках данной работы были разработаны представления, позволяющие упростить получение информации о расписании поездов, продажах билетов и составе бригад.

Первым было создано представление, объединяющие данные из таблиц Prodazhi с таблицей Raspisanie по ключевому полю номера рейса. В результате выборка включает следующие поля: номер рейса, станцию отправления, станцию прибытия, дату и время отправления, дату и время прибытия, а также сумму, вырученную за проданные билеты на каждый рейс Это позволяет получать комплексную информацию о проданных билетах с привязкой к конкретным рейсам. Соответствующая команда представлена в соответствии с рисунком 50.

```
CREATE VIEW itog_summa_prodazh as
SELECT
    p.ID_bileta as идентификатор_билета,
    p.FIO_passazhira as ФИО_пассажира,
    r.Mesto_otprav + ' - ' + r.Mesto_prib AS маршрут,
    p.Stoimost                            AS стоимость_за_1,
    p.Kol_billetov as кол_билетов,
    p.Stoimost * p.Kol_billetov           AS итоговая_сумма
FROM Prodazhi p, Raspisanie r
WHERE p.ID_reysa = r.ID_reysa
```

Рисунок 50 – Создание представления itog_summa_prodazh

Результат работы представления itog_summa_prodazh представлена в соответствии с рисунком 51

```
select * from itog_summa_prodazh
```

100 %

Результаты | Сообщения

	идентификатор_билета	ФИО_пассажира	маршрут	стоимость_за_1	кол_билетов	итоговая_сумма
1	1	Кириллов Иван Степанович	Москва - Владивосток	8500.00	1	8500.00
2	2	Носова Анастасия Олеговна	Санкт-Петербург - Москва	2400.00	2	4800.00
3	3	Дмитриев Кирилл Юрьевич	Москва - Санкт-Петербург	3500.00	1	3500.00
4	4	Романова Виктория Сергеевна	Казань - Москва	4200.00	2	8400.00
5	5	Ершов Александр Борисович	Москва - Новосибирск	4200.00	1	4200.00
6	6	Тимофеева Людмила Аркадьевна	Новосибирск - Омск	8400.00	3	25200.00
7	7	Власов Денис Николаевич	Москва - Екатеринбург	3000.00	1	3000.00
8	8	Гусева Марина Павловна	Екатеринбург - Челябинск	1900.00	2	3800.00

Рисунок 51 – Вывод представления itog_summa_prodazh

Далее было создано представление для подсчёта количества работников по должностям. Результат показан в соответствии с рисунком 52

```
CREATE VIEW kol_rabotnikov_na_dolzhnosti as
SELECT Dolzhnost, COUNT(*) AS количество_работников
FROM Rabotniki
GROUP BY Dolzhnost;
```

Рисунок – 52 Создание представления kol_rabotnikov_na_dolzhnosti

Результат работы представления kol_rabotnikov_na_dolzhnosti представлена в соответствии с рисунком 52

```
select * from kol_rabotnikov_na_dolzhnosti
```

100 %

Результаты | Сообщения

	Dolzhnost	количество_работников
1	Диспетчер	4
2	Кассир	5
3	Машинист	5
4	Механик	2
5	Начальник поезда	3
6	Проводник	6

Следующее представление предназначено для расчёта сумм продаж с учётом льгот. Оно объединяет данные из таблиц продаж, информации о

пассажирах и категориях льгот. Оно позволяет рассчитать общую сумму продаж отдельно для каждой льготной категории. В запросе используются агрегатная функция SUM, а также группировка по типу льготы. Благодаря этому представлению аналитики могут быстро оценить долю льготных продаж в общей выручке без необходимости каждый раз писать сложные группирующие запросы Результат его создания показан в соответствии с рисунком 53.

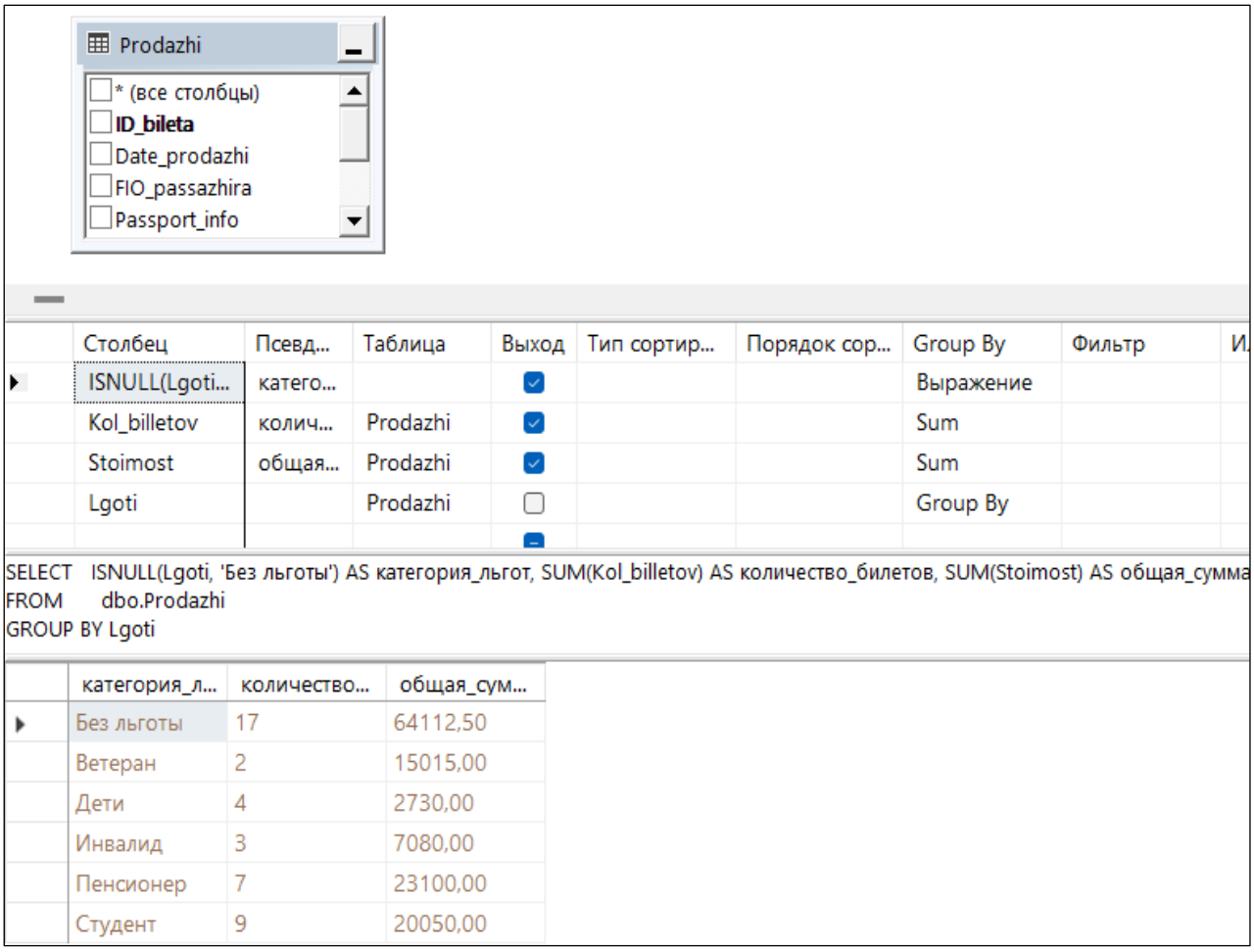

Рисунок 53 – Создание представления summa_po_lgotam

Последнее представление отображает соответствие бригад типам поездов. Оно создаётся на основе внутреннего соединения (INNER JOIN) таблиц Brigadi и Poezda по полю ID_poezda. После создания представление позволяет пользователям обращаться к нему как к виртуальной таблице, выполняя простые запросы: Результат показан в соответствии с рисунком 54.

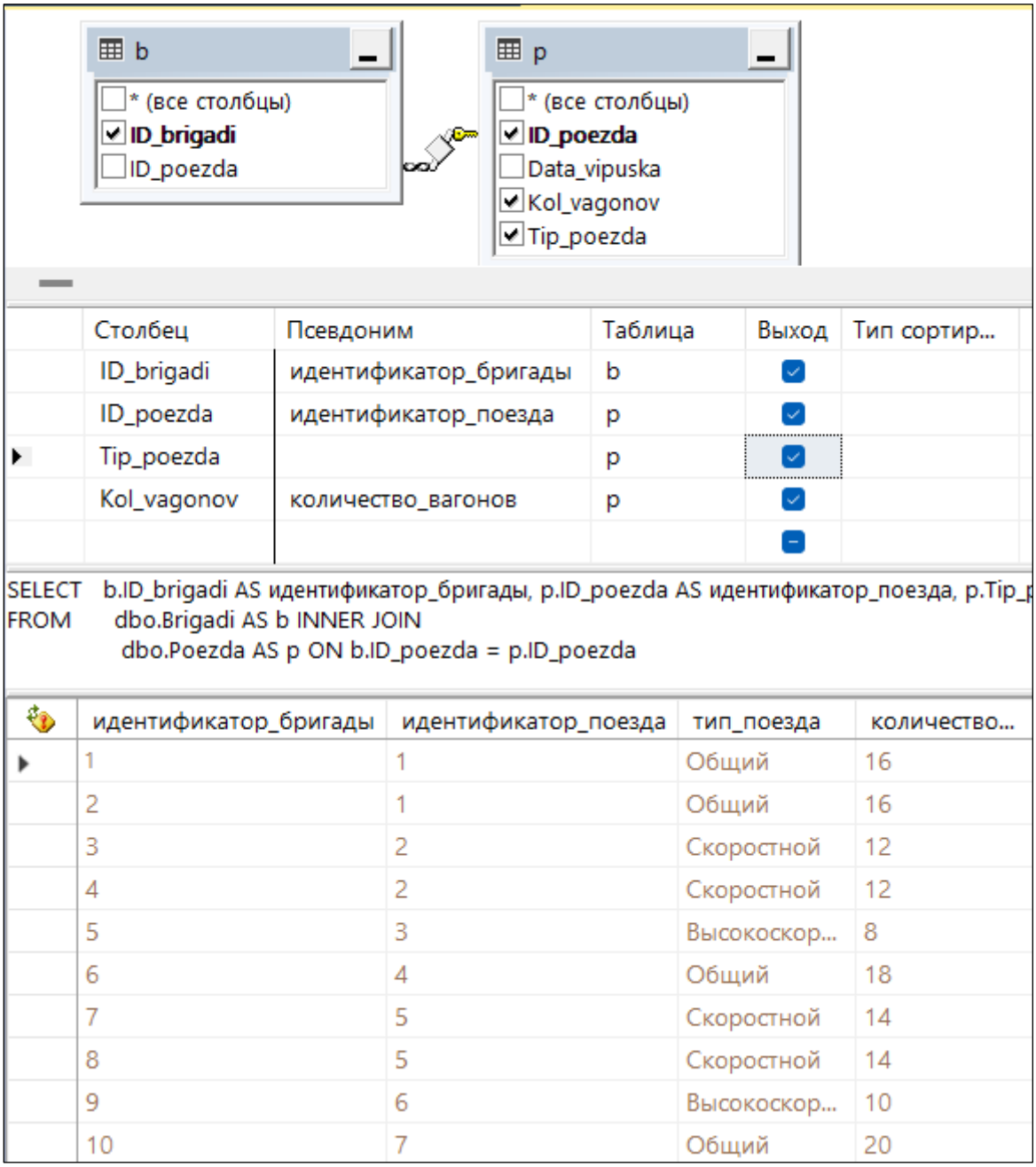

Рисунок 54 – Создание представления brigadi_k_tipu_poezda

В результате выполнения данного этапа были успешно разработаны представления, позволяющие значительно упростить работу с базой данных. Созданные представления обеспечивают удобный доступ к наиболее востребованной информации, объединяя данные из нескольких таблиц и представляя их в структурированном и понятном виде.

Заключение

В процессе реализации курсового проекта был проведён полный цикл проектирования базы данных «Железнодорожный вокзал», начиная с анализа предметной области и заканчивая физической реализацией в Microsoft SQL Server. На этапе анализа были выделены основные сущности (работники, поезда, расписание, бригады, продажи билетов, состав бригад), определены их атрибуты и связи между собой. Построенные инфологическая и даталогическая модели позволили формализовать предметную область без привязки к конкретной СУБД, а затем корректно спроецировать её на реляционную структуру с определением первичных и внешних ключей, типов данных и ограничений целостности, включая вычисляемый стаж сотрудников и проверки допустимых значений.

В среде Microsoft SQL Server Management Studio с использованием T–SQL была создана база данных «ZheleznodorozhniyVokzal», реализованы механизмы резервного копирования и восстановления (BACKUP DATABASE и RESTORE DATABASE), что гарантирует сохранность данных и возможность быстрого восстановления при сбоях. Для разграничения прав доступа разработана ролевая модель: созданы роли admin, kassir и dispetcher с соответствующими пользователями и привилегиями (GRANT, ALTER ROLE). Администратор получил полные права на все таблицы, кассир – доступ только к операциям с билетами и расписанием, диспетчер – к управлению поездами, бригадами и расписанием, что обеспечивает безопасность и разделение обязанностей сотрудников вокзала.

Разработаны шесть таблиц (Rabotniki, Poezda, Brigadi, Raspisanie, Sostav_Brigad, Prodazhi) с ограничениями PRIMARY KEY, FOREIGN KEY, CHECK и UNIQUE, а также построена диаграмма базы данных, наглядно отражающая все связи между таблицами. Таблицы заполнены тестовыми данными, подтверждающими корректность структуры и работы ограничений.

Созданы запросы различных типов: простые запросы на выборку с операторами сравнения, логическими операторами (AND, OR, NOT), проверкой на принадлежность множеству и диапазону, соответствие шаблону и обработкой NULL – значений, что демонстрирует гибкость T–SQL для фильтрации данных. Реализованы многотабличные запросы с использованием декартова произведения, соединения по равенству, INNER JOIN, LEFT JOIN, RIGHT JOIN, а также симметричного соединения таблицы с самой собой, что позволяет объединять данные из нескольких таблиц (продажи – расписание – поезда) для получения комплексной информации о рейсах, бригадах и выручке. Выполнены запросы с агрегатными функциями , встроенными временными функциями, группировкой, условием отбора групп и сортировкой. Также созданы запросы на изменение данных: INSERT, UPDATE и DELETE , что обеспечивает полный цикл управления информацией.

Созданы представления, упрощающие доступ к данным и повышающие уровень безопасности: итоговая сумма продаж по рейсам, количество работников по должностям, сумма продаж по льготным категориям, соответствие бригад типам поездов. Представления скрывают сложную логику запросов от конечных пользователей и предоставляют удобные интерфейсы для анализа деятельности вокзала. Таким образом, поставленная цель курсового проекта полностью достигнута, все задачи выполнены. Разработанная база данных может служить основой для автоматизации ключевых бизнес–процессов железнодорожного вокзала в условиях цифровой трансформации транспортной инфраструктуры.

Поставленная цель достигнута, задачи выполнены.

Список использованных источников

1. Волк, В. К. Базы данных. Проектирование, программирование, управление и администрирование : учебник для вузов / В. К. Волк. — 5-е изд., стер. — Санкт-Петербург : Лань, 2025. — 244 с. — ISBN 978-5-507-53648-1. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/493991
2. Котлинский, С. В. Разработка моделей предметной области автоматизации : Учебник для СПО / С. В. Котлинский. — Санкт-Петербург : Лань, 2021. — 412 с. — ISBN 978-5-8114-8036-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/183206
3. Базы данных и основы языка SQL : учебное пособие / составитель С. В. Коломийцева. — Хабаровск : ДВГУПС, 2023. — 89 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/433529
4. Microsoft. Справочник по Transact-SQL (T-SQL) — Текст : электронный // Microsoft Learn. — URL: https://learn.microsoft.com/ru-ru/sql/t-sql/language-reference
5. Microsoft. Документация по SQL Server Management Studio (SSMS) — Текст : электронный // Microsoft Learn. — URL: https://learn.microsoft.com/ru-ru/sql/ssms/sql-server-management-studio-ssms
6. Microsoft. Резервное копирование и восстановление баз данных SQL Server — Текст : электронный // Microsoft Learn. — URL: https://learn.microsoft.com/ru-ru/sql/relational-databases/backup-restore/back-up-and-restore-of-sql-server-databases
7. Microsoft. Субъекты безопасности: роли и пользователи SQL Server — Текст : электронный // Microsoft Learn. — URL:

https://learn.microsoft.com/ru-ru/sql/relational-databases/security/authentication-access/principals-database-engine

8. ОАО «РЖД». Стратегия развития до 2030 года — Текст : электронный // Официальный сайт ОАО «РЖД». — URL: https://company.rzd.ru/ru/9353